

Statistické metody (entropy encoding)

Zdeněk VAŠÍČEK

Fakulta informačních technologií, Vysoké učení technické v Brně

Brno, Czech Republic

vasicek@fit.vutbr.cz

Statistické metody komprese dat

- **Tunstallovo kódování**
 - Princip, algoritmus tvorby blokových kódů
- **Shannon-Fanovo kódování**
 - Princip, algoritmus, vlastnosti
- **Huffmanovo kódování**
 - Princip, algoritmus, vlastnosti (best a worst case), střední délka kódování
 - Huffmanovy kódy s nejmenší variancí
 - Efektivní implementace kodéru a dekodéru (kanonický Huffmanův kód a jeho tvorba)
 - N-ární Huffmanovo kódování
 - Adaptivní Huffmanovo kódování (FGK algoritmus, Vitterův algoritmus)
- **Aritmetické kódování**
 - Adaptivní aritmetické kódování
 - Intervalové kódování (Range encoding)
- **Asymmetric Numeral Systems (ANS)**

Statistické metody komprese dat (tzv. entropické kodéry)

- Statistické metody komprese dat ke své činnosti využívají znalost pravděpodobnosti výskytu symbolů zdrojové abecedy a tvoří základ většiny kompresních algoritmů.
 - entropický kodér tvoří typicky poslední úroveň kompresního algoritmu
 - výsledkem transformace, predikce či kvantizace jsou hodnoty, které je nutné zakódovat
- Většina metod nahrazuje symboly fixní délky kódovými slovy proměnné délky. Kódová slova jsou volena tak, aby jejich délka rostla proporcionálně k hodnotě $\log_D 1/p$, kde D odpovídá počtu symbolů kódové abecedy a p odpovídá pravděpodobnosti výskytu symbolu vstupní abecedy.
 - Často se vyskytující symboly jsou tím pádem kódovány kratšími kódy.
 - Čím přesněji známe rozložení pravděpodobností, tím vyšší bude účinnost komprese.
- Základní metody:
 - Huffmanovo kódování
 - Aritmetické kódování
 - Intervalové kódování (Range encoding)
- Pokročilé metody:
 - Asymmetric numeral systems (metoda kombinující výhody Huffmanova a aritmetického kódování)

Statistické modelování

Pro určení pravděpodobnosti je možné uvažovat různě komplexní modely

- **Četnosti výskytů symbolů**

- předpokládá se, že jednotlivé výskyty symbolů jsou nezávislé a mají stejné rozdělení
- četnosti se typicky zjistí z bloku dat určité délky

- **Jednoduchá kontextová závislost**

- předpokládá se závislost na k předchozích symbolech – viz Markovův model k -tého řádu

- **Pokročilá kontextová závislost**

- použití variabilního počtu předchozích symbolů jako kontextu (PPM - Prediction by Partial Matching)
- kombinování predikcí z více kontextů (CM - Context Mixing)

- **Znamé rozdělení**

- pokud data respektují jisté rozdělení pravděpodobnosti, je možné určit pouze parametry tohoto rozdělení a používat pravděpodobnosti z tohoto modelu. Viz např. Normální rozdělení, Exponenciální rozdělení, Cauchyho rozdělení, atd. (viz kódování čísel)

Jakmile známe pravděpodobnosti výskytu symbolů, můžeme využít faktu, že

- symbol a s pravděpodobností výskytu p má informační obsah $i(a) = \log_2 \frac{1}{p}$ bitů
- kód délky r implikuje pravděpodobnost výskytu $P(a) = 2^{-r}$

Tunstallovo kódování (blokové entropické)

- Většina entropických kodérů typicky kóduje symboly pevné délky proměnným počtem bitů. Výjimku tvoří Tunstallův kód (1967), který používá kódová slova pevné délky přičemž kódové slovo reprezentuje různě dlouhý řetězec symbolů.
 - Kód je odolnější vůči chybě při přenosu oproti kódům s proměnnou délkou kódových značek.
 - Práce s pevným počtem bitů kódových slov je jednodušší.
- Motivace: Uvažujme vstup tvořený symboly z abecedy $\Sigma = \{A, B\}$. Zdroj dat může produkovat sekvence, které často obsahují podřetězce AA, AAA, AB, AAB, ale málo kdy se vyskytne podřetězec BB.
 - Můžeme použít kódování, kdy $AA \rightarrow 000, AAA \rightarrow 001, AB \rightarrow 010, ABA \rightarrow 011, a B \rightarrow 100$ (tzn. variable-to-fixed).
 - Málo pravděpodobný výskyt BB bude kódován jako 100100, pokud bude předcházet A, použije se kratší kód 010, 011 nebo 100.
 - Problém: proces kódování není jednoznačný (AAABAAB, lze interpretovat jako AAA, B, AA, B, nebo jako AA, ABA, AB), kódování je neefektivní (využito pouze 5 z 8 kombinací) a nereflexuje četnosti výskytů jednotlivých znaků.
- Řetězce symbolů vstupní abecedy tvoří tzv. slovník.
 - Pro dostatečně velký slovník se průměrný počet bitů na symbol blíží entropii zdroje.
 - Slovník je nutné konstruovat tak, abychom byli schopni kódovat libovolný vstupní tok.
 - Kódová slova lze přiřadit položkám slovníku libovolně.
 - Nevýhoda: slovník musí být uložen do výstupního toku, aby bylo možné vstup rekonstruovat.

Tunstallův kód

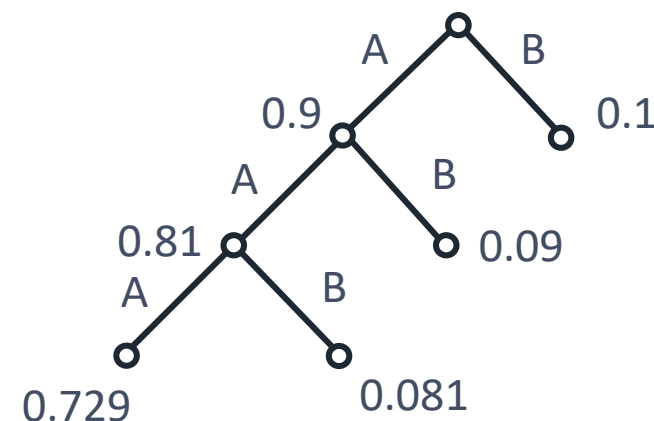
■ Algoritmus:

- Vstup: zdrojová abeceda Σ , pravděpodobnosti jednotlivých symbolů $P(a)$, $a \in \Sigma$, a N definující max. velikost kódové abecedy. Předpokládá se zdroj generující na sobě nezávislé symboly se stejným rozdělením pravděpodobností, tj. $P(a_1 a_2 \dots a_k) = P(a_1)P(a_2) \dots P(a_k)$
- Výstup: slovník T kterému se libovolně přiřadí kódy o délce N bitů, tj. $f: T \rightarrow \{0,1\}^N$

```

 $T \leftarrow \Sigma$ 
 $i \leftarrow 0$ 
 $n \leftarrow |T|$ 
while  $n + (i + 1)(n - 1) \leq 2^N$  do
     $x \leftarrow w \in T, P(w) \geq P(w'), w' \in T$ 
     $T \leftarrow (T \setminus \{x\}) \cup \{xy | y \in \Sigma\}$ 
     $i \leftarrow i + 1$ 

```



Příklad: $\Sigma = \{A, B\}$, $P(A) = 0.9$, $P(B) = 0.1$, $N = 2$.

Výstup: $T = (AAB, AB, B, AAA)$ a

$P(t) = (0.081, 0.09, 0.1, 0.729)$, $t \in T$;

$\langle f \rangle = 2 / (3(0.081 + 0.729) + 2 * 0.09 + 0.1) =$

0.738 bitů na symbol; $H(\mathcal{S}) = 0.468$

■ Střední délka kódování je rovna

$$\langle f \rangle = \frac{N}{\sum_{w \in T} P(w) |w|}$$

Shannon-Fanovo kódování

- Shannon-Fanovo kódování (1948) je první metoda navržená pro nalezení *optimálních prefixových kódů* pracující na principu rozkladu množiny na podmnožiny.
- Algoritmus:
 - Vstup: zdrojová abeceda $\Sigma = \{a_1, \dots, a_n\}$, $n \geq 2$, uspořádaná dle pravděpodobnosti výskytu jednotlivých symbolů $P(a_i)$ tak, aby $P(a_i) \geq P(a_j)$ pro $i < j$
 - Výstup: $f: \Sigma \rightarrow \{0,1\}^+$

```
def SF(l, r):
    if l + 1 = r then
        f(al) ← 0
        f(ar) ← 1
        return
    j ← argminj |Σi=lj P(ai) - Σi=j+1r P(ai)|
    SF(l, j)
    SF(j + 1, r)
    f(ai) ← 0f(ai) for i = l ... j
    f(ai) ← 1f(ai) for i = j + 1 ... r
```

a _i	p(a _i)	1	2	3	4	Code
a ₁	0.36	0	00			00
a ₂	0.18		01			01
a ₃	0.18	1	10			10
a ₄	0.12		11	110		110
a ₅	0.09			111	1110	1110
a ₆	0.07				1111	1111

Příklad: $\Sigma = \{a_1, \dots, a_6\}$, tj. SF(1,6)

$$\langle f \rangle = 2(0.36 + 0.18 + 0.18) + 3 \cdot 0.12 + 4(0.09 + 0.07) = 2.44, H = 2.37$$

Shannon-Fanovo kódování

- Algoritmus pro konstrukci Shannon-Fanova kódování produkuje optimální kód pouze za určitých podmínek.
 - Shannon-Fanova metoda dává dobrý kód, když jsou rozklady dobré, tj. když podmnožiny v každém rozkladu mají velmi blízké výsledné pravděpodobnosti.
Např. případ, kdy pravděpodobnosti jsou záporné mocniny 2.
- Příklad: $\Sigma = \{A, B, C, D, E\}$, zpráva obsahující 39 symbolů $freq(a) = \{15, 7, 6, 6, 5\}$, $a \in \Sigma$
 - $H = 2.19$ bitů na symbol
 - Shannon fanovo kódování:
 $f_{SC}(a) = \{00, 01, 10, 110, 111\}$,
 $\langle f_{SC} \rangle = \frac{2}{39}(15 + 7 + 6) + \frac{3}{39}(6 + 5) = 2.28$ bitů na symbol
 - Huffmanovo kódování:
 $f_{HC}(a) = \{0, 100, 101, 110, 111\}$,
 $\langle f_{HC} \rangle = \frac{1}{39}(15) + \frac{3}{39}(7 + 6 + 6 + 5) = 2.23$ bitů na symbol

a	$freq(a)$	1	2	3
A	15	0	00	
B	7		01	
C	6	1	10	
D	6		11	110
E	5			111

Huffmanovo kódování

- Huffmanovo kódování (Huffman, 1952) je všeobecně používaná metoda komprese dat, která tvoří základ mnoha aplikací (ZIP, JPEG, MP3, ...).
- Huffmanova metoda se podobá Fano-Shannonově metodě, ale dává obecně lepší kód.
 - Podobně jako Shannon-Fanova metoda řeší problém nalezení **optimálního prefixového kódu** pro abecedu symbolů a dané pravděpodobnostní rozdělení.
 - Hlavní rozdíl mezi těmito dvěma metodami je, že Shannon-Fano metoda konstruuje kódová slova shora dolů (od levého nejvyššího bitu k nejnižšímu), kdežto Huffman konstruuje kódová slova zdola nahoru.
 - Metoda dává optimální kód pro pravděpodobnosti symbolů rovné záporným mocninám 2.
- **Huffmanův algoritmus byl předmětem intenzivního výzkumu v oblasti komprese dat.**
 - Důkaz optimálnosti a neexistence jiného kódování s kratší průměrnou délkou (Gilbert and Moore, 1959)
 - Odvození horní hranice redundance (Gallager, 1978). Pro běžný text má Huffmanův kód redundanci $R = 0.1$ bitů na symbol oproti ideálnímu entropickému kodéru jako je např. Aritmetické kódování.
 - Varianta Huffmanova kódování s omezenou délkou značek (Fraenkel, 1993)

Horní hranice redundance (Gallager, 1978)

■ Horní hranice redundance

$$R = \begin{cases} p_{max} + 0.086 & \text{pokud } p_{max} \geq 0.5 \\ p_{max} & \text{pokud } p_{max} < 0.5 \end{cases}, \text{ kde } p_{max} \text{ je pravděpodobnost nejčastějšího symbolu.}$$

■ Vliv velikost abecedy na efektivitu Huffmanova kódování (p_{max}):

- Početná abeceda má tendenci mít malé p_{max} => menší redundance
- Malá abeceda má tendenci mít velké p_{max} => větší redundance

■ R lze libovolně přiblížit 0, pokud budeme vstup zpracovávat po blocích o n symbolech, kde jsme odvodili:

$$H(S) \leq L < H(S) + \frac{1}{n}$$

- $n=1$ odpovídá Shannonově větě o kódování
- počet symbolů však roste exponenciálně s n a je roven D^n , což je nepraktické

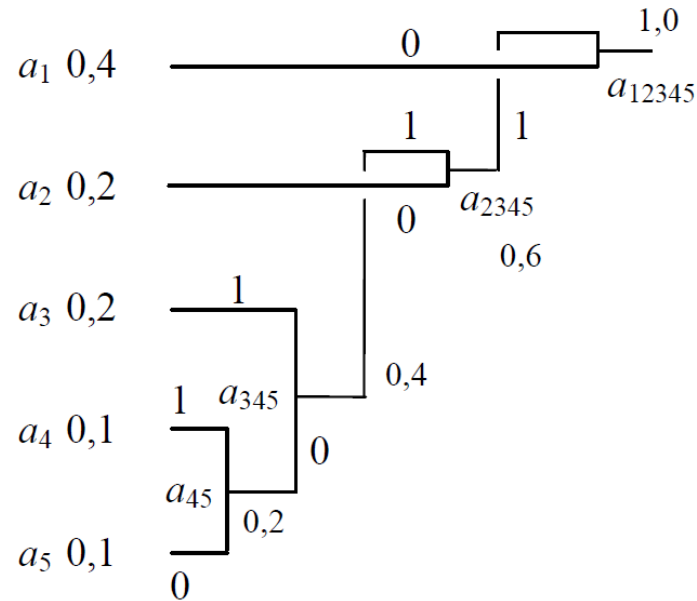
Huffmanovo kódování – algoritmus konstrukce binárního Huffmanova stromu

- **Konstrukce Huffmanova kódování je rekurzivní proces, který**
 - přiřazuje symbolům s vyšší pravděpodobností výskytu kratší značky a dvěma symbolům (binární kódová abeceda) vyskytujícím se nejméně často značky stejné délky
 - začíná s dvěma (binární kódová abeceda) nejméně se vyskytujícími symboly

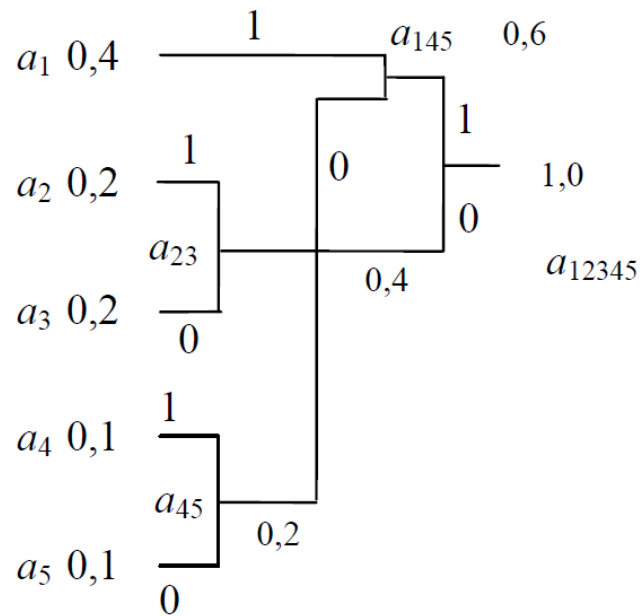
- **Princip (neformálně):**
 - Metoda začíná vytvořením seznamu abecedních symbolů v sestupném pořadí jejich pravděpodobností.
 - Pak se konstruuje strom, se symbolem v každém listu a to zdola nahoru. To se provádí v krocích, kdy se v každém kroku vyberou dva symboly s nejnižší pravděpodobností, sečtou se do vrcholu místního sbíhání větví, vyškrtnou se ze seznamu a nahradí se přidáním symbolem, který je oba reprezentuje. Když se seznam redukuje na jediný symbol, který reprezentuje celou abecedu, je strom dokončen.
 - Strom se pak systematicky prochází a tím se zjistí kódy pro všechny symboly.

Huffmanovo kódování – konstrukce binárního Huffmanova stromu

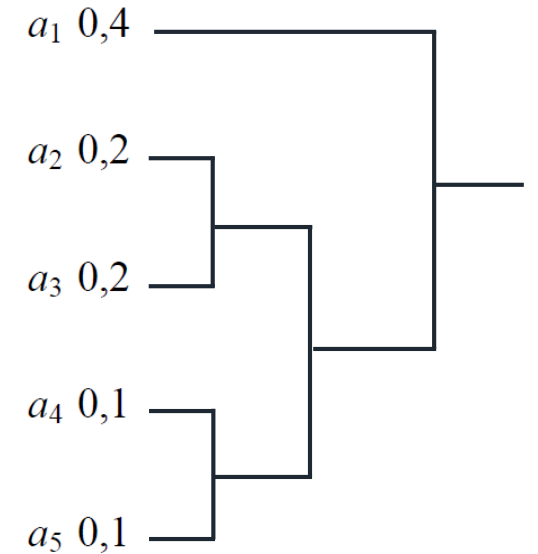
- Příklad: $\Sigma = \{a_1, a_2, a_3, a_4, a_5\}$, $P(a_i) = \{0.4, 0.2, 0.2, 0.1, 0.1\}$
 - Huffmanův kód není jediný, ukázka tří různých stromů:



$$f_{HC1}(a_i) = \{0, 10, 111, 1101, 1100\}$$



$$f_{HC2}(a_i) = \{11, 01, 00, 101, 100\}$$



$$f_{HC3}(a_i) = \{1, 011, 010, 000, 001\}$$

- Střední délka:

$$\begin{aligned} - \langle f_{HC1} \rangle &= \frac{4}{10} \cdot 1 + \frac{2}{10} \cdot 2 + \frac{2}{10} \cdot 3 + \frac{1}{10} \cdot 4 + \frac{1}{10} \cdot 4 = 2.2 \text{ bitů/symbol,} \\ - \langle f_{HC2} \rangle &= \frac{4}{10} \cdot 2 + \frac{2}{10} \cdot 2 + \frac{2}{10} \cdot 2 + \frac{1}{10} \cdot 3 + \frac{1}{10} \cdot 3 = 2.2 \text{ bitů/symbol,} \\ - \langle f_{HC3} \rangle &= \frac{4}{10} \cdot 1 + \frac{2}{10} \cdot 3 + \frac{2}{10} \cdot 3 + \frac{1}{10} \cdot 3 + \frac{1}{10} \cdot 3 = 2.2 \text{ bitů/symbol.} \end{aligned}$$

Optimalita Huffmanova kódování

- **Věta:** Prefixové kódování s proměnnou délkou kódových slov je optimální tehdy, když splňuje následující podmínky:
 1. Máme-li dva symboly a_j a a_k a $P(a_j) \geq P(a_k)$, pak pro délku kódových slov platí $l_j \leq l_k$
 2. Dva symboly s nejmenší pravděpodobností mají stejnou maximální délku kódových slov l_m .
 3. Ve stromové reprezentaci kódování musí z každého vnitřního uzlu vycházet dvě větve.
 - jinak můžeme odebrat při zachování podmínky jednoznačné dekódovatelnosti => snížení průměrné délky
 4. Pokud změníme vnitřní uzel na listový vytvořením nového symbolu, který vznikne zkombinováním listových uzlů příslušejícím k tomuto uzlu, pak platí, že pokud původní strom reprezentoval optimální kódování, tak také nový strom reprezentuje optimální kódování avšak pro redukovanou abecedu.
 - Pokud by tato podmínka neplatila, lze zkombinovaný symbol reprezentovat kódem s menší průměrnou délkou. Takový kód bychom pak mohli rozbalit a nahradit symbol novým stromem. To je však v rozporu s tvrzením, že původní strom reprezentuje optimální kódování.
- **Důsledek:**
 - Podmínka 3, 2 a 1 říká, že kódová slova délky l_m přiřazená dvěma symbolům vyskytujícím se s nejmenší pravděpodobností se liší pouze v posledním bitu. Podmínka 4 říká, že můžeme zkombinovat tyto dva symboly, čímž dostaneme redukovanou abecedu. Pak ale můžeme stejný princip (redukce abecedy) zopakovat znovu aniž bychom porušili některou z podmínek. To je však princip konstrukce Huff. k.

Huffmanovo kódování – konstrukce kódových slov pro libovolnou abecedu

■ Algoritmus:

- Vstup: zdrojová abeceda $\Sigma = \{a_1, \dots, a_n\}$, pravděpodobnosti výskytu jednotlivých symbolů $P(a_i)$, kódová abeceda $\Gamma = \{b_1, \dots, b_m\}$
- Výstup: $f: \Sigma \rightarrow \Gamma^+$

```

def HC( $\Sigma' = \{a'_1, \dots, a'_{n'}\}, \{p'_1, \dots, p'_{n'}\}$ ):
    seřad' prvky  $\Sigma'$  tak, aby  $p'_i \geq p'_j$  pro  $i < j$ 
    if  $n' \leq m$  then
         $f(a'_i) \leftarrow b_i$  for  $i = 1, \dots, n'$ 
    else
        if  $n' = n$  then
             $m' \leftarrow (n - 2) \bmod (m - 1) + 2$ 
        else
             $m' \leftarrow m$ 
        HC( $\Sigma' \setminus \{a'_{n'-m'+2}, \dots, a'_{n'}\}, \{p'_1, \dots, p'_{n'-m'}, \sum_{i=n'-m'+1}^{n'} p'_i\}$ )
         $f(a'_{n'-m'+i}) \leftarrow f(a'_{n'-m'+i}) b_i$  for  $i = 1, \dots, m'$ 

```

Příklad:

$\Sigma = \{a_1, a_2, a_3, a_4, a_5\}$, $\Gamma = \{0, 1\}$,
 $P(a_i) = \{0.4, 0.2, 0.2, 0.1, 0.1\}$

$HC(\Sigma, P)$

$m = 2, n' = 5, n = 5$,

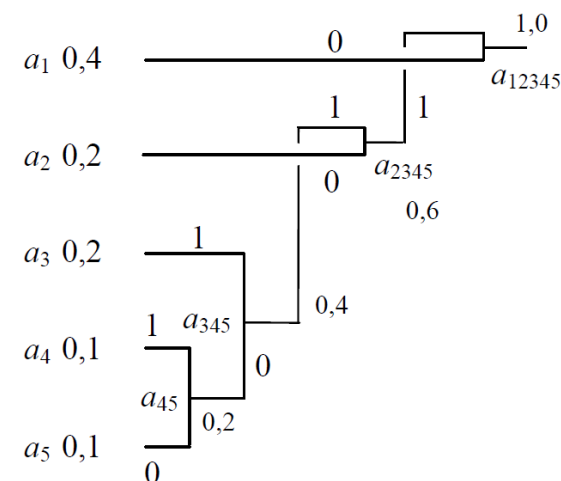
$\Sigma' = \{a'_1, \dots, a'_5\} = \{a_1, a_2, a_3, a_4, a_5\}$

$m' = 3 \bmod 1 + 2 = 2$

$n' - m' + 2 = 5$

$\Rightarrow HC(\{a'_1, a'_2, a'_3, a'_4\}, \{0.4, 0.2, 0.2, 0.2\})$

$f(a'_4) = f(a'_4) 0, f(a'_5) = f(a'_5) 1$

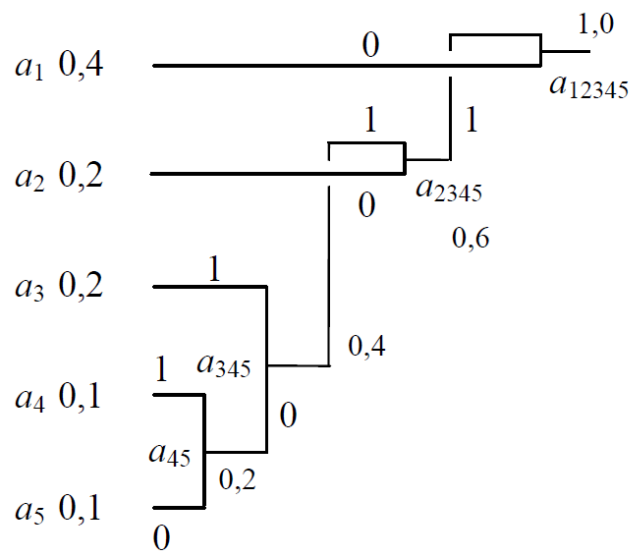


Vztah střední délky a vah uzlů Huffmanova stromu

- Dá se ukázat, že v Huffmanově stromu (obecně stromu, kde je hodnota každého uzlu součtem hodnot svých potomků) se váhový součet listů, kde váhy jsou vzdálenosti listů od kořene, rovná součtu vah vnitřních uzlů.
 - Tato vlastnost poskytuje způsob výpočtu střední délky množiny Huffmanových kódů bez násobení. Jednoduše se sečtou hodnoty všech vnitřních uzlů stromu (včetně kořenového).

■ Příklad:

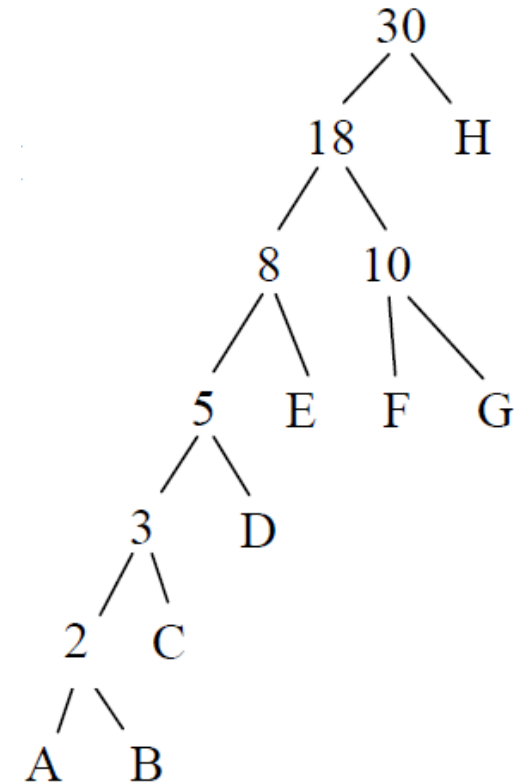
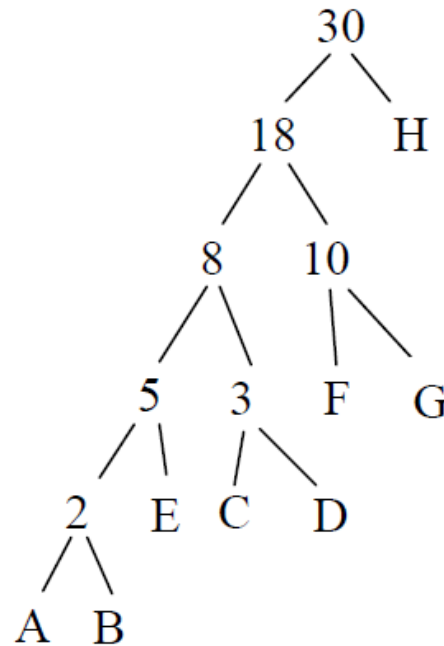
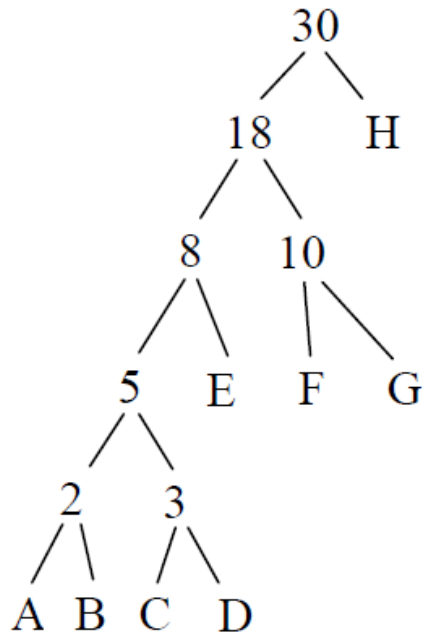
- $\langle f \rangle = \sum_{i=1}^5 p_i l_i = \frac{4}{10} \cdot 1 + \frac{2}{10} \cdot 2 + \frac{2}{10} \cdot 3 + \frac{1}{10} \cdot 4 + \frac{1}{10} \cdot 4 = 2.2$ bitů/symbol
- $\langle f \rangle = a_{45} + a_{345} + a_{2345} + a_{12345} = 0.2 + 0.4 + 0.6 + 1.0 = 2.2$ bitů/symbol



Huffmanovo kódování – konstrukce Huffmanova stromu

■ Příklad:

- Je dáno osm symbolů $\Sigma = \{A, B, C, D, E, F, G, H\}$ s pravděpodobnostmi $P(a) = \frac{1}{30}\{1,1,1,2,3,5,5,12\}$.
- Nakreslete tři různé Huffmanovy kódy s výškami 5 a 6 pro tyto symboly a vypočtete střední délku kódu pro každý z nich.



Huffmanovy kódy s nejmenší variancí

- Jsou různé Huffmanovy kódy pro danou množinu symbolů ekvivalentní nebo má některý Huffmanův kód výhodnější vlastnosti?
 - Ukazuje se, že libovolná rozhodnutí, učiněná při konstrukci Huffmanova stromu ovlivňují jednotlivé kódy, ale ne střední délku kódu nebo redundanci (viz též důkaz optimálnosti Huff. kódování).
 - Kromě střední hodnoty však můžeme zkoumat další parametry – např. rozptyl (angl. variance) kódování.
- **Variance kódování** určuje, jak moc se délky jednotlivých kódů odchylují od střední délky.

$$Var(f) = \sum_a (|f(a)| - \langle f \rangle)^2$$

- Huffmanův kód s minimální variancí je výhodnější v případech, kdy kodér vysílá komprimovaný tok komunikační linkou.
 - Kód s velkou variancí způsobí, že kodér generuje bity s proměnnou rychlostí => vyrovnávací paměť. Čím je variance kódu vyšší, tím je vyšší kolísání rychlosti vstupu bitů do vyrovnávací paměti, což vyžaduje, aby kodér použil delší vyrovnávací paměť.
 - Huffmanův kód s nulovou variancí bude vstupovat do vyrovnávací paměti s konstantní rychlostí, takže bude stačit krátká vyrovnávací paměť.
 - Když kodér pouze zapisuje komprimovaný tok do souboru, nehraje variance kódu žádnou roli.

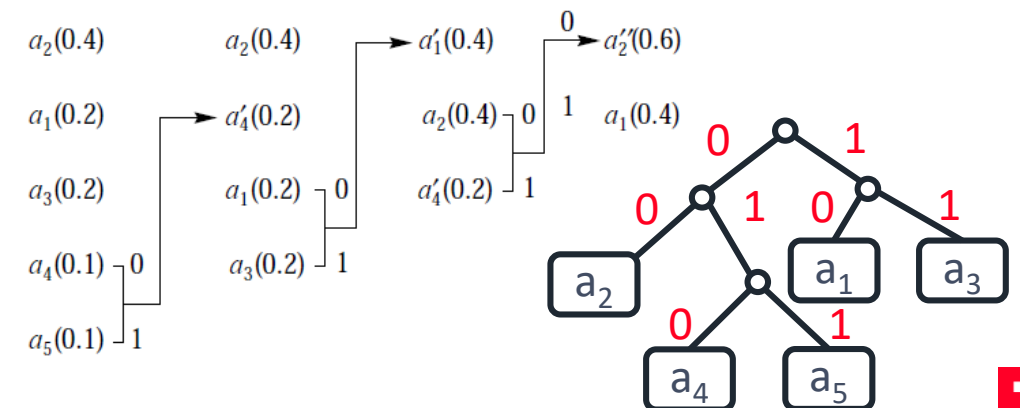
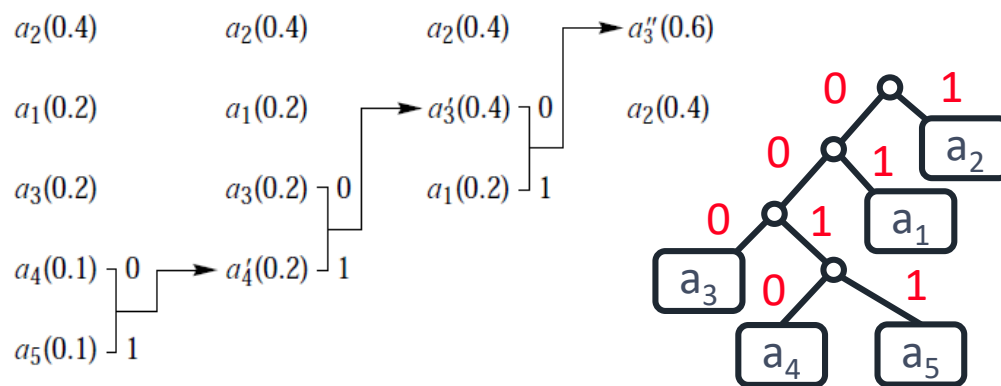
Huffmanovy kódy s nejmenší variancí

■ Variance kódování pro uvedené příklady:

- Pro f_{HC1} je $Var(f_{HC1}) = \frac{4}{10}(1 - 2.2)^2 + \frac{2}{10}(2 - 2.2)^2 + \frac{2}{10}(3 - 2.2)^2 + \frac{1}{10}(4 - 2.2)^2 + \frac{1}{10}(4 - 2.2)^2 = 1.36$ bitů/symbol
- Pro f_{HC2} je $Var(f_{HC2}) = \frac{4}{10}(2 - 2.2)^2 + \frac{2}{10}(2 - 2.2)^2 + \frac{2}{10}(2 - 2.2)^2 + \frac{1}{10}(3 - 2.2)^2 + \frac{1}{10}(3 - 2.2)^2 = 0.16$ bitů/symbol.

■ Tvorba *Huffmanova kódování s minimální variancí*: Existuje-li možnost volby více než jednoho symbolu se stejnou pravděpodobností, zvol takové dva, které se ve stromu nachází co nejbližše listovým uzlům.

- Modifikace řadicí procedury: Původní a'_n , bude po seřazení na pozici a'_i , kde $i < j$ pro všechna j , pro která platí, že $p'_j = p'_i$

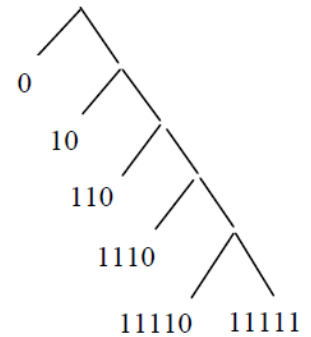
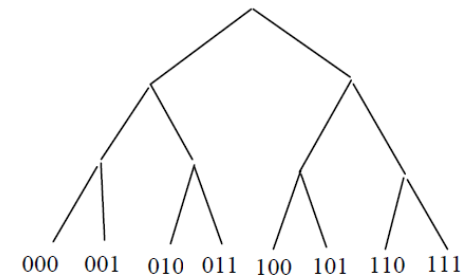


Vlastnosti Huffmanova kódování pro binární kódovou abecedu ($m=2$)

- Pokud jsou pravděpodobnosti rovny záporným mocninám dvou, tj. $p_i = 2^{-i}$, $i \geq 1$, pak redundance kódování je rovna nule.
- Mají-li dva a více symbolů stejnou pravděpodobnost výskytu, pak způsob volby symbolů v průběhu konstrukce má vliv na výsledný kód.
 - Viz Huffmanovo kódování s minimální variabilitou
- Pokud $p_i \geq \sum_{j=i+2}^n p_j$, $p_i \geq p_j$ pro $i < j$, pak kódové slovo a_i , $i = 1, \dots, n-1$, odpovídá unárnímu kódu hodnoty $i-1$, tj $a_i = \mathbf{1}^{i-1}\mathbf{0}$, a kódové slovo $a_n = \mathbf{1}^n$
 - Příklad: $p_i = 2^{-i}$, $p_n = 2^{-(n-1)}$, $i = 1, \dots, n-1$
 - Pro $i = 1, \dots, 7$ máme $p_i = \{0.5, 0.25, 0.125, 0.0625, 0.03125, 0.015625, 0.015625\}$ a dostáváme $f_{\text{HC}}: a_1 \rightarrow 0, a_2 \rightarrow 10, a_3 \rightarrow 110, a_4 \rightarrow 1110, a_5 \rightarrow 11110, a_6 \rightarrow 111110, a_7 \rightarrow 111111$
- Pokud $p_1 < p_{n-1} + p_n$, $p_i \geq p_j$ pro $i < j$, pak kódové slovo a_i , $i = 1, \dots, 2^{k+1} - n$, odpovídá blokovému kódu délky $k = \lfloor \log_2 n \rfloor$ a délky $k+1$ pro $i = 2^{k+1} - n + 1, \dots, n$
 - Příklad: $p_i = 1/n$
 - Pro $i = 1, \dots, 5$ máme $p_i = \{0.2, 0.2, 0.2, 0.2, 0.2\}$, tj. $k = 2$ a dostáváme $f_{\text{HC}}: a_1 \rightarrow 00, a_2 \rightarrow 01, a_3 \rightarrow 10, a_4 \rightarrow 111, a_5 \rightarrow 110$

Vlastnosti Huffmanova kódování pro binární kódovou abecedu ($m=2$)

- Výška kódového stromu generovaného Huffmanovým algoritmem udává délku nejdelšího kódu.
 - Podmínky vedoucí na nejnižší a nejvyšší Huffmanův strom jsou uvedeny na předchozím slide.
- Nejnižší Huffmanův strom se vytvoří v případě, že symboly mají stejné pravděpodobnosti.
 - Strom je vyvážený, nebo téměř vyvážený a jeho výška je $\lceil \log_2 n \rceil$.
 - Ve speciálním případě, kdy je počet symbolů n mocninou 2 je výška přesně $\log_2 n$.
- Nejvyšší Huffmanův strom se získá v případě, že pravděpodobnosti vedou v každém kroku na zvýšení stromu o jedna.
 - Takový strom se nazývá nevyvážený či nesouměrný (angl. unbalanced, lopsided).
 - Výška nevyváženého stromu bude rovna $n - 1$ (v každém kroku zkombinujeme symbol získaný v předchozím kroku s dalším symbolem).



Nevyvážený Huffmanův strom

- Nevyvážený Huffmanův strom maximální výšky vznikne v případě, že pravděpodobnosti tvoří Fibonacciho posloupnost.

- Odvození:

- Dvě nejmenší pravděpodobnosti označíme $p_1 = a$ a $p_2 = b$. Z principu konstrukce Huffmanova kódu plyne, že symboly musí mít přiřazeny pravděpodobnosti následovně:

$$p_3 = p_1 + p_2 = a + b, \quad p_4 = b + (a + b) = p_2 + p_3, \quad p_5 = (a + b) + (a + 2b) = p_3 + p_4, \text{ atd.}$$

- Posloupnost odpovídá Fibonacciho posloupnosti $F_{n+1} = F_{n-1} + F_n$, jejíž první dva členy jsou $F_0 = a, F_1 = b$.

- Příklad:

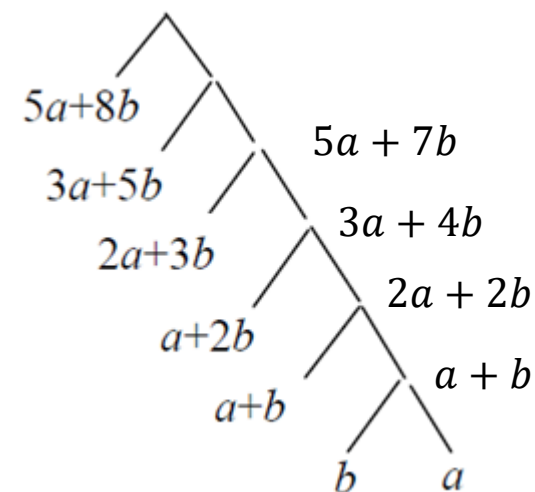
- Nechť $F_{1...9} = \{1, 5, 6, 11, 17, 28, 45, 73, 118\}$, $\Sigma = \{a_1, \dots, a_9\}$,

$$P(a_i) = \frac{1}{304} \{1, 5, 6, 11, 17, 28, 45, 73, 118\}$$

- $f_{\text{HC}}: a_1 \rightarrow 0, a_2 \rightarrow 10, a_3 \rightarrow 110, a_4 \rightarrow 1110, a_5 \rightarrow 11110,$
 $a_6 \rightarrow 111110, a_7 \rightarrow 1111111, a_8 \rightarrow 11111100, a_9 \rightarrow 11111101$

- Řešení:

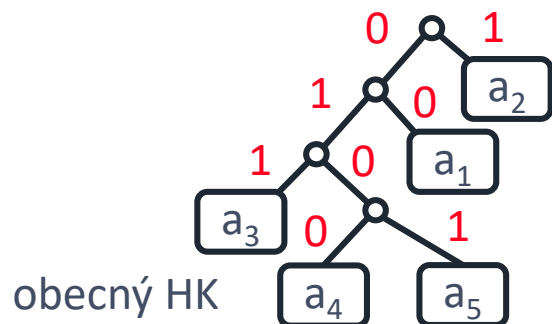
- přetvarovat strom mírnou změnou pravděpodobností symbolů



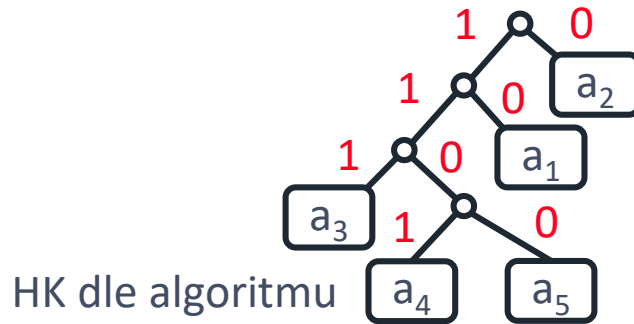
Kanonický Huffmanův kód

- Pro danou abecedu a pravděpodobnosti existuje mnoho různých Huffmanových kódů.
- Počet různých binárních Huffmanových kódových stromů pro n symbolů odpovídá 2^{n-1}
 - Binární Huffmanův strom pro n symbolů má $n - 1$ vnitřních uzlů. Z každého vnitřního uzlu vycházejí dvě hrany ohodnocené 0 a 1. Prohozením těchto dvou hodnot vznikne jiný Huffmanův kódový strom.
- Algoritmus konstruuje jeden z Huffmanových kódů
 - Systematicky přiřazuje méně pravděpodobnému symbolu hodnotu 0 (typicky levá větev stromu) a více pravděpodobnému hodnotu 1
- V praxi se používá tzv. *kanonický Huffmanův kód*, který existuje pro dané pravd. jediný.
 - Kratší kódová slova mají numericky menší hodnotu (alternativně lze uvažovat i vyšší), než slova delší
 - Kódová slova stejných délek jsou volena tak, aby numerické hodnoty tvořily postupně rostoucí sekvenci

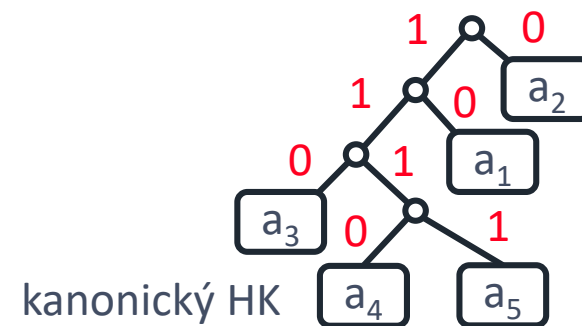
Příklady Huffmanových kódů pro $p(a_i) = \{0.2, 0.4, 0.2, 0.1, 0.1\}$:



obecný HK



HK dle algoritmu

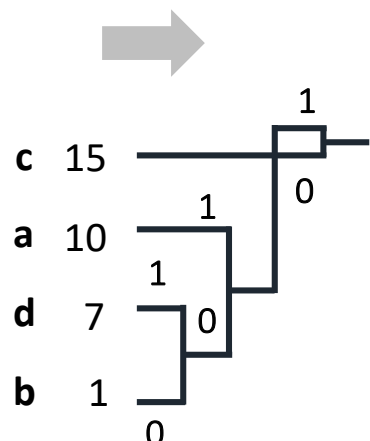


kanonický HK

Kanonický Huffmanův kód (f_{CHC}) – konstrukce

- Konstrukce kanonického Huffmanova kódu, kde **kratší slova mají nižší num. hodnotu**
 - Nejprve zkonstruujeme Huffmanův strom a pro každý symbol a_i zjistíme délku kódové značky l_i
 - Seřadíme symboly vzestupně dle délky kódové značky, získáme posloupnost délek $l_1 \leq l_2 \leq \dots \leq l_n$
 - Kódová slova přiřazujeme následovně: Prvnímu symbolu přiřadíme kódové slovo 0^{l_1} , tj. hodnotu $c_1 = 0$ reprezentovanou pomocí l_1 bitů. Dalšímu symbolu přiřadíme kódové slovo, které bude kódovat hodnotu $c_2 = (c_1 + 1) \ll \Delta_l$, kde $\Delta_l = l_2 - l_1$. Obecně i -tému symbolu ($i \geq 2$) přiřadíme kódové slovo odpovídající hodnotě $c_i = (c_{i-1} + 1)2^{l_i - l_{i-1}}$.
- Příklad:

SYMBOL	ČETNOST
a	10
b	1
c	15
d	7



HUFFMANŮV KÓD	DÉLKA l_i
11	2
100	3
0	1
101	3

SYMBOL	DÉLKA l_i	Δ_l	c_i	KANONICKÝ HUFF. KÓD
c	1	0	0	0
a	2	1	$(0+1) \ll 1$	10
b	3	1	$(2+1) \ll 1$	110
d	3	0	$(6+1) \ll 0$	111

Kanonický Huffmanův kód (f'_{CHC}) – konstrukce (alternativa)

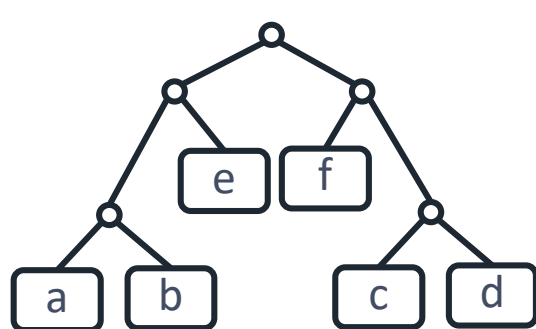
- Konstrukce kanonického Huffmanova kódu, kde **kratší slova mají vyšší num. hodnotu**
 - Nejprve zkonstruujeme Huffmanův strom a pro každý symbol a_i zjistíme délku kódové značky l_i
 - Seřadíme symboly vzestupně dle délky kódové značky, získáme posloupnost délek $l_1 \leq l_2 \leq \dots \leq l_n$
 - Kódová slova přiřazujeme následovně: Poslednímu symbolu přiřadíme kódové slovo 0^{l_n} , tj. hodnotu $c_n = 0$ reprezentovanou pomocí l_n bitů. Dalšímu symbolu přiřadíme kódové slovo, které bude kódovat hodnotu $c_{n-1} = (c_n + 1) \gg \Delta_l$, kde $\Delta_l = l_n - l_{n-1}$. Obecně i -tému symbolu ($i < n$) přiřadíme kódové slovo odpovídající hodnotě $c_i = (c_{i+1} + 1) 2^{l_{i+1} - l_i}$.

■ Příklad:

SYMBOL	ČETNOST		HUFFMANŮV KÓD	DÉLKA L_i		SYMBOL	i	DÉLKA L_i	Δ_l	C_i	KANONICKÝ HUFF. KÓD
a	10		11	2		d	4	3	0	0	000
b	1		100	3		b	3	3	0	$(0+1) \gg 0$	001
c	15		0	1		a	2	2	1	$(1+1) \gg 1$	01
d	7		101	3		c	1	1	1	$(1+1) \gg 1$	1

Kanonický Huffmanův kód – vlastnosti

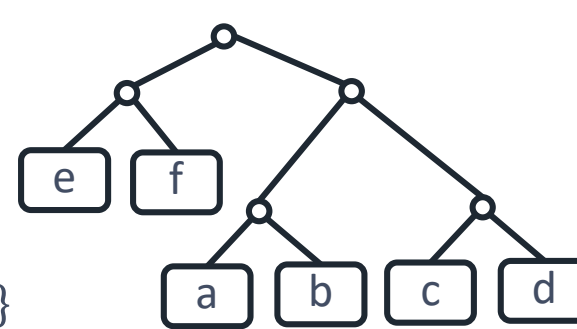
- Kanonický Huffmanův kód není Huffmanovým kódem v pravém smyslu
 - nelze jej získat přímo Huffmanovým algoritmem, nutná dvoustupňová konstrukce
 - díky zachování délek kódových slov má však kanonická forma stejné parametry (redundance, variabilita)



$f_{HC}: a \rightarrow 000, b \rightarrow 001,$
 $c \rightarrow 110, d \rightarrow 111,$
 $e \rightarrow 01, f \rightarrow 10$

$\Sigma = \{a, b, c, d, e, f\}$

$freq(a) = \{10\ 11\ 12\ 13\ 22\ 23\}$



$f_{CHC}: e \rightarrow 00, f \rightarrow 01$
 $a \rightarrow 100, b \rightarrow 101,$
 $c \rightarrow 110, d \rightarrow 111$

$f'_{CHC}: e \rightarrow 10, f \rightarrow 11$
 $a \rightarrow 000, b \rightarrow 001,$
 $c \rightarrow 010, d \rightarrow 011$

- Kanonický Huffmanův kód zvyšuje efektivitu komprese a rychlost dekódování.
 - Do výstupního toku není nutné ukládat symboly a zvolené přiřazení kódových značek (drahé), stačí uložit délky kódových slov.
 - Není nutné používat stromovou strukturu s referencemi (n listů, n-1 interních uzlů, každý uzel pointer na levý a pravý syn a symbol), což může být nevýhodné pro abecedy s velkým počtem symbolů (např. kódování textu po slovech) nejen z pohledu paměťových nároků, ale efektivitu procházení stromové struktury (malá lokalita). Pro symboly kódované stejnou délkou stačí mít uloženu jen první hodnotu a seznam symbolů v lexikografickém pořadí.

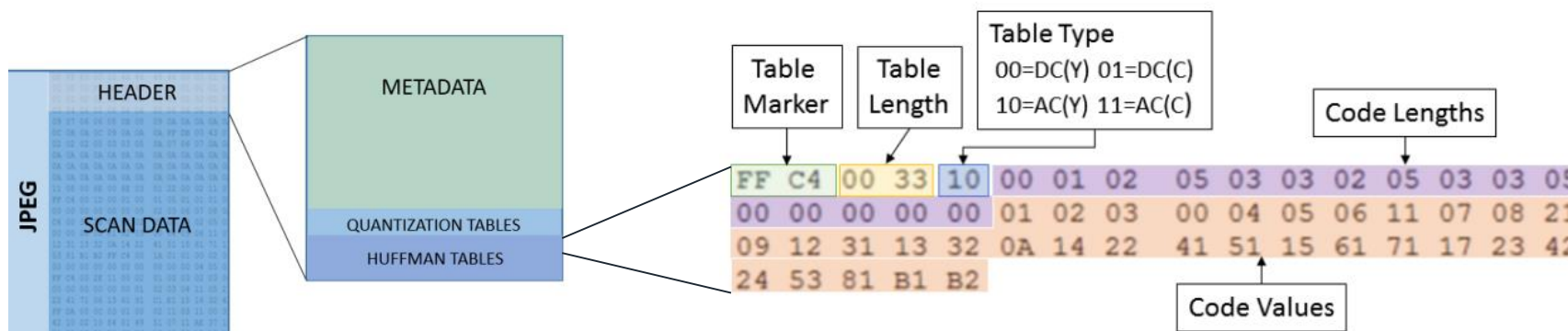
Způsob uložení kódové knihy kanonického Huffmanova kódu

$$f: \Sigma \rightarrow \{0,1\}^+$$

- Jedna z výhod kanonického Huffmanova kódu je možnost využití úspornější reprezentace popisující přiřazení kódových slov jednotlivým symbolům (tzv. kódová kniha, angl. codebook).
- Možnosti uložení kódové knihy u běžného Huffmanova kódu:
 - trojice (symbol, počet bitů kódové značky, kódová značka)
 - dvojice (počet bitů kódové značky, kódová značka) pokud známe předem počet symbolů a ty jsou seřazeny lexikograficky
- V případě kanonického Huffmanova kódu známe postup, jak přiřazovat kódové značky
 - stačí uložit pouze počet bitů, z kterého lze zrekonstruovat kódová slova
- Příklad: mějme Huffmanův kód $f_{HC}: A \rightarrow 11, B \rightarrow 0, C \rightarrow 101, D \rightarrow 100$ a jeho kanonickou verzi $f_{CHC}: A \rightarrow 10, B \rightarrow 0, C \rightarrow 110, D \rightarrow 111$
 - Trojice pro f_{HC} : ('A',2,11), ('B',1,0), ('C',3,101), ('D',3,100)
 - Dvojice pro f_{HC} : (2,11), (1,0), (3,101), (3,100)
 - f_{CHC} : (2,1,3,3)

Způsob uložení kódové knihy kanonického Huffmanova kódu

- V případě, kdy nemáme symboly sekvenčně, je možné použít alternativní způsob reprezentace, kdy uložíme jen ty symboly, které se používají a k nim informaci o délce, tzn. n -tici $L = (l_1, l_2, \dots, l_n)$ následovanou m -ticí $A = (a_1, a_2, \dots, a_m)$
 - Prvek L_i určuje počet symbolů, které mají kódové slovo délky i
 - Prvky A_i jsou symboly z Σ seřazené podle délek, přičemž vynecháme nepoužité symboly
 - Pokud není hodnota n fixní, pak je nutné také uložit
- Příklad pro $f_{CHC}: A \rightarrow 10, B \rightarrow 0, C \rightarrow 110, D \rightarrow 111, n = 3$:
 - $f_{CHC}: (3) (1,1,2) ('B','A','C','D')$
- Tento přístup je aplikován u formátu JPEG, kde se vyskytuje max. 162 symbolů abecedy
 - kódová slova mají maximálně 16 bitů, symboly jsou 8-bitové hodnoty



$l_1 = 0 \quad \{ \}$
 $l_2 = 1 \quad 00 \rightarrow \{01\}$
 $l_3 = 2 \quad 010, 011 \rightarrow \{02, 03\}$
 $l_4 = 5 \quad 1000, 1001, 1010, 1011, 1100 \rightarrow \{00 \text{ (EOB)}, 04, 05, 06, 11\}$

Efektivní dekodér kanonického Huffmanova kódu

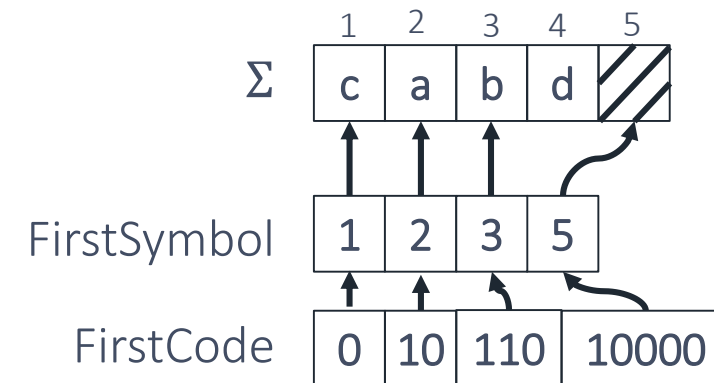
■ Algoritmus (inicializace kódové knihy):

- Vstup: počet kódových slov zakódovaných pomocí i bitů, $L(i) = |\{a \in \Sigma : |f_{CHC}(a)| = i\}|$, kde $i = 1, \dots, N$ a $N = \max_{a \in \Sigma} |f_{CHC}(a)|$
- Výstup: $firstCode(i)$ určující hodnotu prvního kódového slova délky i bitů, $firstSymbol(i)$ určující index prvního slova z $A = \{a_1, \dots, a_m\}$, $a_i \in \Sigma$, uspořádané vzestupně dle hodnoty kódových slov

```
def FCFS( $L = \{l_1, \dots, l_N\}$ ):
     $c \leftarrow 0$ 
     $s \leftarrow 1$ 
    for  $i = 1 \dots N + 1$  then
         $firstCode(i) \leftarrow c$ 
         $firstSymbol(i) \leftarrow s$ 
         $s \leftarrow s + l_i$ 
         $c \leftarrow (c + (l_i - 1) + 1) \ll 1$ 
```

Příklad: $\Sigma = \{c, a, b, d\}$, $L(i) = \{1, 1, 2\}$
 Viz příklad pro $freq(a) = \{10, 1, 15, 7\}$,
 $f_{CHC}: a \rightarrow 10, b \rightarrow 110, c \rightarrow 0, d \rightarrow 111$

i	First code	First symbol	s	c
1	0 (0)	1	2	2
2	2 (10)	2	3	6
3	6 (110)	3	5	16
4	16 (10000)	5		



Efektivní dekodér kanonického Huffmanova kódu

■ Algoritmus (dekodér):

- Vstup: $firstcode$, $firstsymbol$, Σ uspořádaná vzestupně dle hodnoty kódových slov, vstupní tok
- Výstup: dekódovaná sekvence

```
def FastCHD(input, output):
```

```
    c ← 0
```

```
    l ← 0
```

```
    while not eof do
```

```
        l ← l + 1
```

```
        c ← (c << 1) + readnextbit(input)
```

```
        if (c << 1) < firstcodel+1 then
```

```
            putsymbol(output,  $\Sigma_{firstsymbol[l]+c-firstcode[l]}$ )
```

```
            l ← 0
```

```
            c ← 0
```

Příklad:

$\Sigma = \{c, a, b, d\}$,

$firstcode(i) = \{0, 2, 6, 16\}$

$firstsymbol(i) = \{1, 2, 3, 5\}$

Viz příklad pro $freq = \{10, 1, 15, 7\}$,

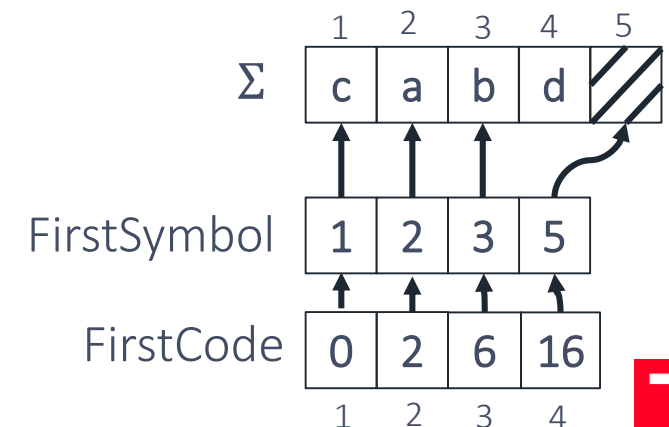
$f: a \rightarrow 10, b \rightarrow 110, c \rightarrow 0, d \rightarrow 111$

Vstup: 110 110 0 111 10

l : 1, 2, 3, 1, 2, 3, 1, 1, 2, 3, 1, 2

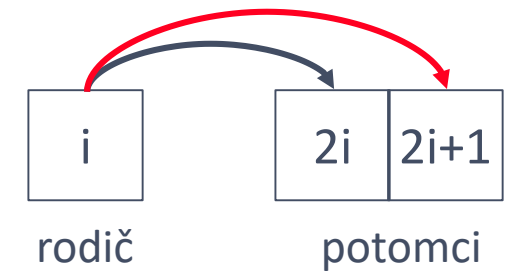
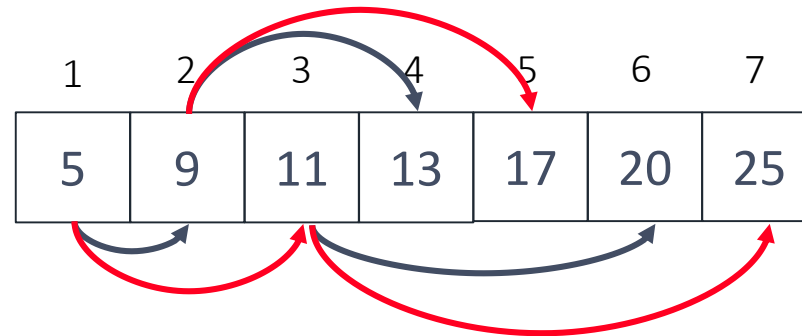
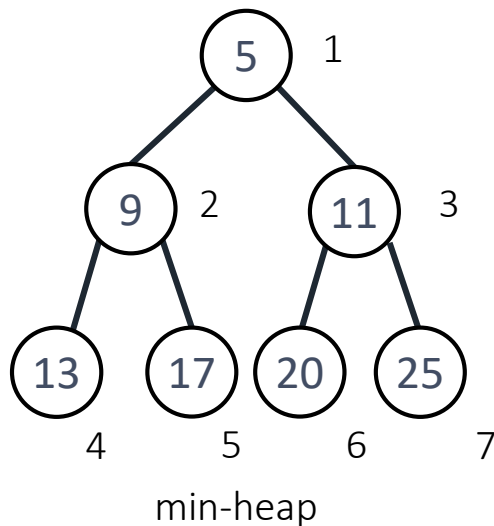
c : 1, 3, 6, 1, 3, 6, 0, 1, 3, 7, 1, 2

 b b c d a



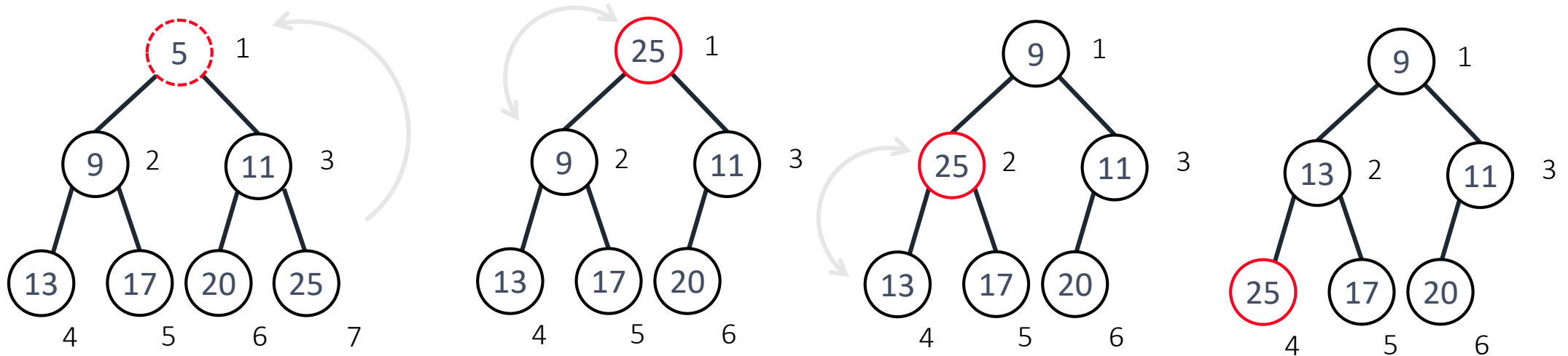
Výpočet kanonického Huffmanova kódu

- Kodér se typicky implementuje pomocí datové struktury *heap* (*hromada*, *halda*), kterou lze efektivně implementovat pomocí pole pevné délky.
- *Heap* je binární strom, ve kterém každý vnitřní uzel je asociován s dvojicí (klíč, datová položka) a uzly jsou uspořádány tak, že při průchodu jakoukoliv cestou z listu do kořenu jsou klíče uzlů na cestě v neklesajícím (max-heap) nebo nerostoucím (min-heap) pořadí.
 - Pro každý uzel platí, že jeho klíč má hodnotu větší nebo rovnu hodnotě klíče rodiče (min-heap)
 - Heap může být reprezentován bez použití ukazatelů vektorem tak, že každý uzel uložený na indexu i má svého levého potomka na indexu $2i$ a pravého potomka na indexu $2i + 1$. Rodič uzlu j je na pozici $\lfloor j/2 \rfloor$.



Datová struktura heap

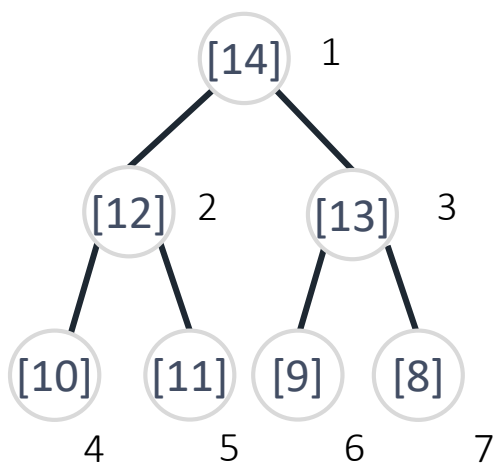
- Základní operací je odstranění kořene (extractMin, pop) a přeskládání zbývajících uzlů tak, abychom znovu dostali heap. Přeskládání se nazývá prosívání (angl. *sifting*).
- Princip operace prosívání:
 - Prosívání začne přesunutím nejnižšího uzlu napravo do funkce nového kořene.
 - Kořen se porovná se svými potomky a případně se s některým z nich musí prohodit, aby bylo dodrženo uspořádání. Prohození může být zapotřebí více, aby se obnovila relace uspořádání. Nejvyšší počet prohození je roven výšce stromu, což je $h = \lfloor \log_2 n \rfloor$.
- Příklad:



Výpočet délek kódových slov Huffmanova kódování

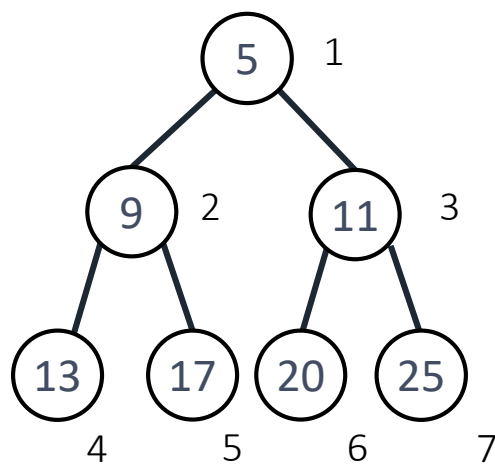
- Algoritmus (Hirschberg – Sieminski) vypočítává délky kódů abecedy s n symboly s použitím jediného pole o velikosti $2n$.
 - Do horní poloviny (pozice $n + 1$ až $2n$) se uloží frekvence výskytu všech n symbolů
 - Spodní polovina pole (pozice 1 až n) se používá jako *min-heap*, přičemž datové položky jsou ukazatele (realizované indexy) na příslušné hodnoty frekvencí uložené v horní polovině.

1	2	3	4	5	6	7	8	9	10	11	12	13	14
14	12	13	10	11	9	8	25	20	13	17	9	11	5



klíč

min-heap

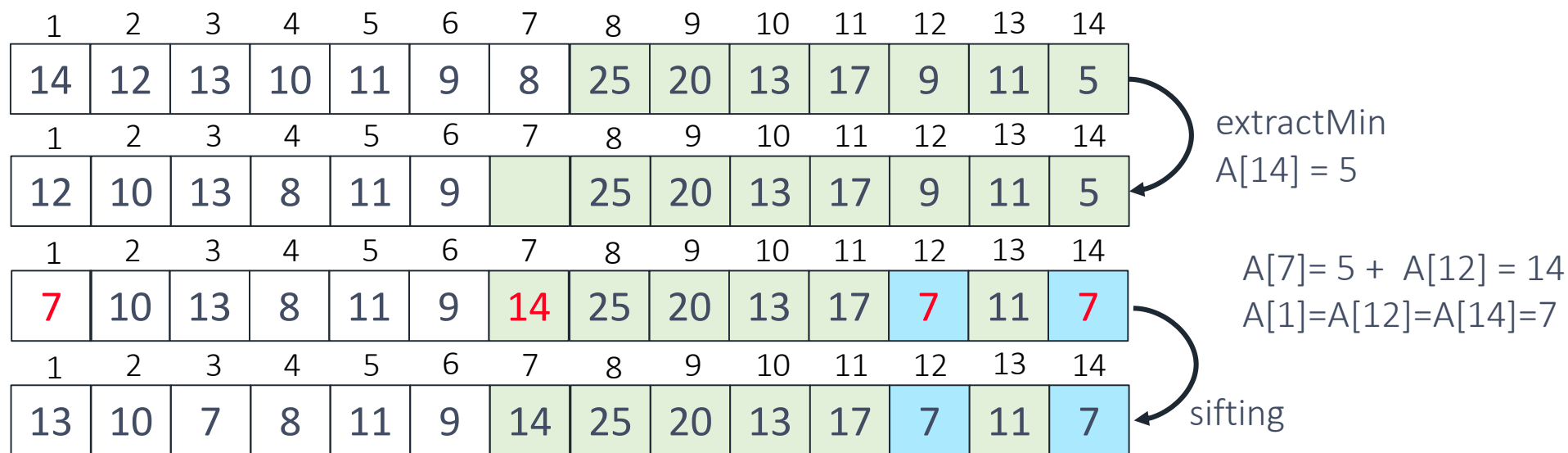


hodnota

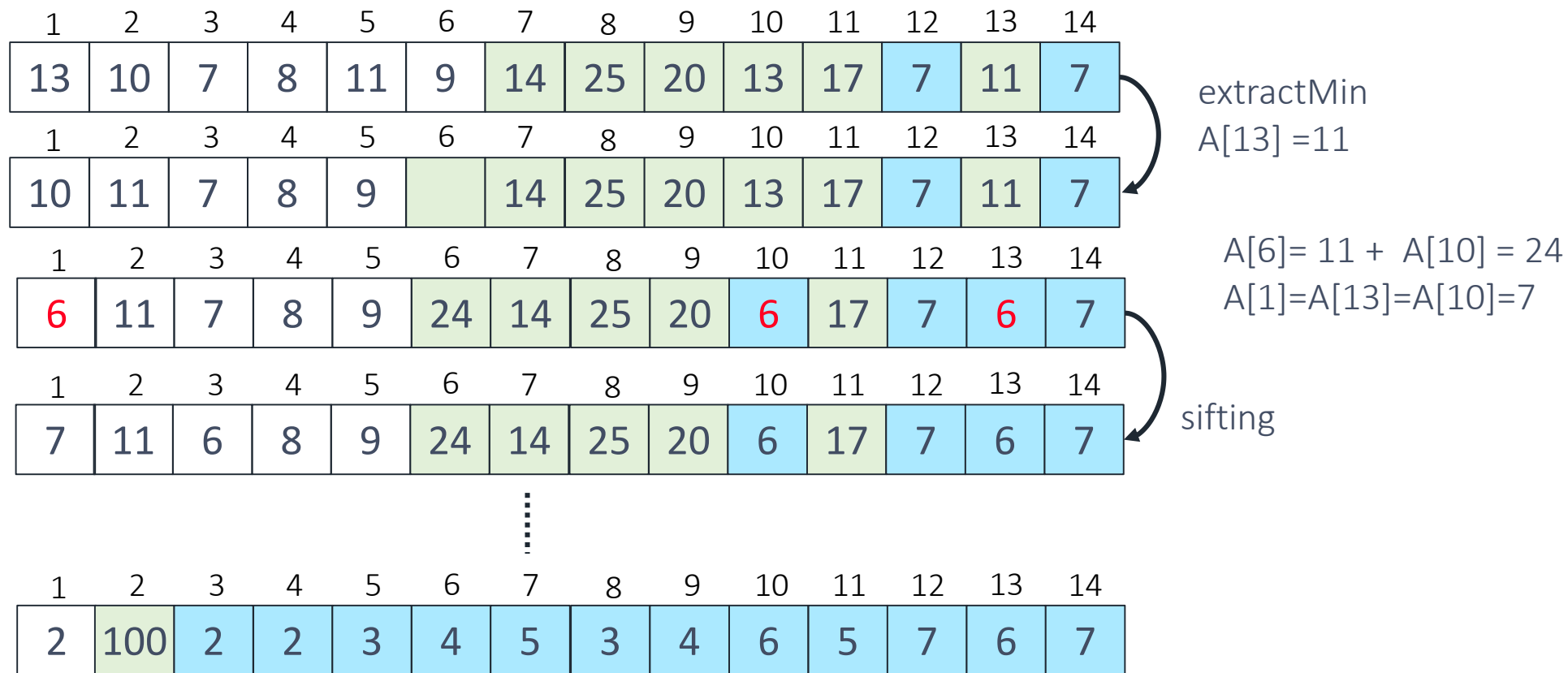
Výpočet délek kódových slov Huffmanova kódování

■ Princip:

- V každé iteraci se použije *heap* k nalezení dvou nejnižších frekvencí a jejich náhradě součtem. Součet je uložen v poslední pozici hromady $A[h]$, ukazatele nahradí dvě nejnižší frekvence a hromada se zmenší o jednu pozici. Cyklus probíhá tak dlouho, dokud se hromada neredukuje na jediný ukazatel.



Výpočet délek kódových slov Huffmanova kódování



- Na konci tvoří heap jediná položka $A[0]=2$ s počtem 100. Délky kódových značek se určí pomocí průchodu ukazatelů odzadu.
 - Např. symbol 14: $A[14]=7 \Rightarrow A[7]=5 \Rightarrow A[5]=3 \Rightarrow A[3]=2 = \text{root} = 4 \text{ bity}$; $A[8]=3 \Rightarrow A[3]=2 = \text{root} = 2 \text{ bity}$

Výpočet délek kódových slov Huffmanova kódování

■ Algoritmus

```
def HClen(freq):
    m = len(freq)
    h, hr, bitlen = [], [None]*(2*m), [0]*m

    for i, f in enumerate(freq): #init heap
        heapq.heappush(h, (f, m + i))

    while m>1: #perform Hirschberg-Sieminski
        a = heapq.heappop(h)
        b = heapq.heappop(h)
        m = m - 1
        hr[a[1]] = hr[b[1]] = m
        heapq.heappush(h, (a[0] + b[0], m))

    for i in range(len(freq)): #determine code lengths
        j = hr[len(freq)+i]
        l = 1
        while j>1:
            j = hr[j]
            l += 1
        bitlen[i] = l

    return bitlen
```

- Dává algoritmus kódování s minimální variancí?

■ Příklad:

Vstup: freq = [10,1,15,7], symb = ['a','b','c','d']

h=[(1, 5), (7, 7), (15, 6), (10, 4)]

m= 4

a (1, 5) b (7, 7)

h [(8, 3), (15, 6), (10, 4)]

hr [None, None, None, None, None, 3, None, 3]

m= 3

a (8, 3) b (10, 4)

h [(15, 6), (18, 2)]

hr [None, None, None, 2, 2, 3, None, 3]

m= 2

a (15, 6) b (18, 2)

h [(33, 1)]

hr [None, None, 1, 2, 2, 3, 1, 3]

Výstup: bitlen [2, 3, 1, 3]



Princip komprese binárních dat pomocí Huffmanova kódu

■ Algoritmus:

- Abychom nemuseli ukládat délku souboru, používá se speciální symbol EOF
- Výstup doplníme do násobku osmi bitů např. nulami

```
def compressHC(infile, bitout):  
    #určení počtu výskytů jednotlivých ASCII hodnot 0-255  
    freqs = get_frequencies(infile)  
    #přidání EOF symbolu s počtem výskytů 1  
    freqs[256] = 1  
  
    enc = HuffmanEncoder(bitout)  
    #výpočet délek, sestavení kanonického kódu  
    enc.codetree = CanonicalHuffmanCode(freqs)  
  
    #uložení max. 256 8-bitových délek, tj. 2048 bitů!  
    enc.write_code_len_table()  
  
    #kódování symbol po symbolu  
    while not eof:  
        b = infile.read(1)  
        enc.write(ord(b))  
    enc.write(256) # EOF
```

Praktické problémy HK

- Vstupní abeceda obsahující malý počet symbolů
 - Řešení: rozšířený HK
- Délka kódových značek
 - Řešení: truncated HK
- Latence a lokální koncentrační vlastnost dat
 - Řešení: adaptivní HK

Rozšířený Huffmanův kód

- Průměrná délka Huffmanova kódování je shora ohraničena hodnotou $H(X) + p_{max} + 0.086$ (Gallager, 1978)
 - Dle Shannonovy věty platí $H(X) \leq L(X) \leq H(X) + p_{max} + 0.086$, což zaručuje dobré výsledky
- Nicméně s klesajícím počtem symbolů vstupní abecedy účinnost metody klesá
 - Mějme např. $P = (0.8, 0.18, 0.02)$ a $f_{HC} = (0, 10, 11)$, pak $L = |f_{HC}| = 1.2$,
 $H(X) = 0.816 < L < H(X) + 0.8 + 0.086$ a $R = 0.384$ (38.4%)
 - Extrémní případ je $\Sigma = \Gamma = \{0, 1\}$, kde nedosahujeme žádné komprese třebaže stále platí uvedený vztah
- Řešením je kódovat vstupní symboly po n -ticích, tzn. uvažujeme rozšířený zdroj X^n
 - Příklad: Pokud budeme pro $P = (0.8, 0.18, 0.02)$ kódovat dvojice symbolů, pak máme abecedu velikosti 9 prvků s $L = 1.7228/2 = 0.8614$ a $R = 0.045$ (4.5%)
- Máme-li abecedu A , pak rozšířená abeceda řádu m odpovídá

$$A^m = \underbrace{A \times A \times \cdots \times A}_{m\text{-krát}} = \{ \underbrace{a_1 a_1 \dots a_1}_{m \text{ symbolů}}, \underbrace{a_1 a_1 \dots a_1 a_2}_{m \text{ symbolů}}, \dots, a_n a_n \dots a_n \}$$

- kde $A \times A$ je kartézský součin A . Pokud uvažujeme nezávislé jevy se stejným rozložením pravděpodobnosti, pak $P(a_i^m) = \prod_{j=1}^m P(a_{ij}^m)$. Získáme tedy n^m nových symbolů.

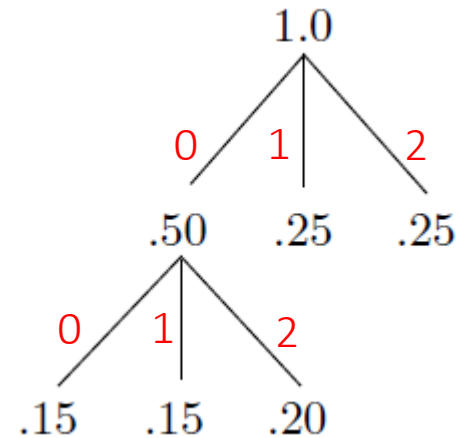
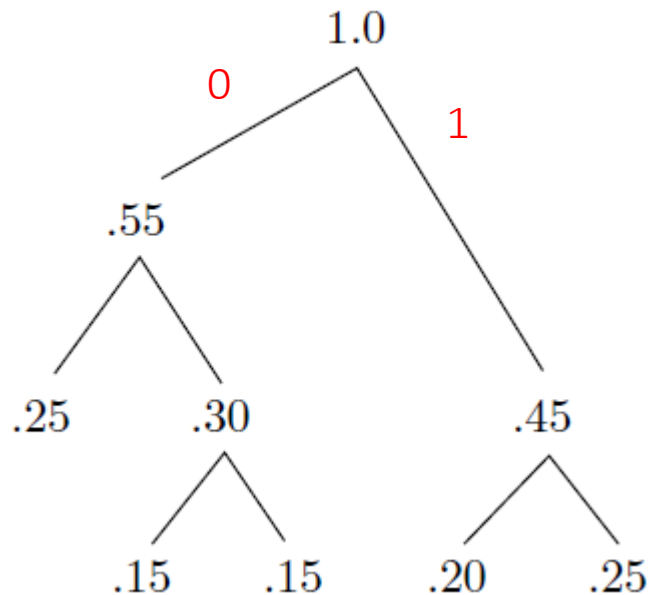
Alternativní řešení: n-ární Huffmanovo kódování

- Huffmanovo kódování není omezeno pouze na binární abecedu a lze jej snadno aplikovat i na kódové abecedy s více než dvěma prvky.
 - Přenosová média jsou typicky analogového charakteru a dovolují vyjádřit více než dvě hodnoty (např. různá úroveň amplitudy, různá fáze sinusovek, ...). Komunikace může být efektivnější, pokud máme k dispozici více kódových symbolů. Výhodné např. pro zvýšení přenosové kapacity komunikačních linek (optické, bezdrátové spojení).
- Místo dvou symbolů s nejmenší pravděpodobností nahrazujeme m' symbolů (viz algoritmus HC)
 - m' odpovídá počtu symbolů kódové abecedy $|\Gamma| = m$ s výjimkou první iterace, kde $2 \leq m' \leq m$ volíme tak, abychom v dalších iteracích měly vždy k dispozici požadovaný počet symbolů.
 - Příklad: Pokud $m=3$ a máme 6 symbolů, tak po první iteraci a odebrání 3 symbolů bychom měli $6-3+1=4$ symboly. Po druhé $4-3+1=2$ a poslední iteraci už není možné dokončit. Proto v první iteraci vezmeme 2 symboly.

n-ární Huffmanovo kódování

■ Příklad ternárního kódování:

- Nechť $P(i) = \{0.15, 0.15, 0.2, 0.25, 0.25\}$, pak
- $F_{HC} = 2 \times 0.25 + 3 \times 0.15 + 3 \times 0.15 + 2 \times 0.20 + 2 \times 0.25 = 2.3$ bits/symbol.
- $F_{3HC} = 2 \times 0.15 + 2 \times 0.15 + 2 \times 0.20 + 1 \times 0.25 + 1 \times 0.25 = 1.5$ trits/symbol



Huffmanovy kódy omezené délky

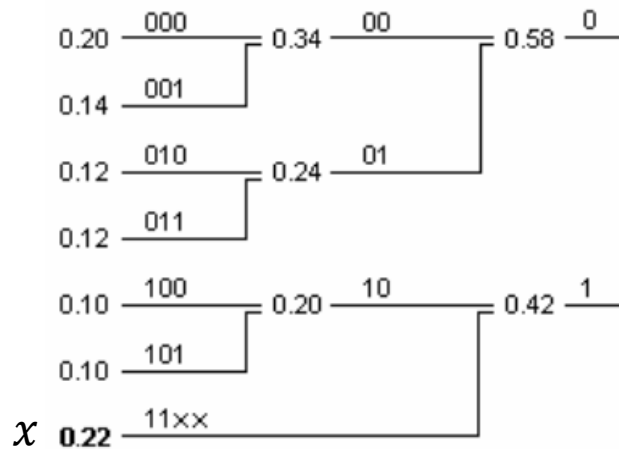
- V praxi se setkáváme s potřebou omezit maximální délku kódových značek
 - Nejdelší kódové slovo může být pro abecedu s n symboly $n - 1$ bitů dlouhé. Velké abecedy (např. kódování po slovech) by vedly na dlouhá kódová slova a zvýšené nároky na uchování kódové knihy.
- Důvody:
 - Snaha omezit velikost kódové knihy.
 - Díky odhadu rozložení z omezeného množství dat mohou být některé pravděpodobnosti podhodnocené a blízké nule.
 - Omezit maximální počet kroků potřebných na dekódování. Má vliv na propustnost a je zásadní např. v oblasti multimédií a telekomunikací.
 - Praktická implementace Hirschberg-Sieminsky (256-bitový datový typ pro 8-bitové symboly)
- Přístupy:
 - Truncated Huffman coding (suboptimální)
 - Hirschberg-Larmore: A fast algorithm for optimal length-limited Huffman codes 1990

Golin, Zhang A Dynamic Programming Approach To Length-Limited Huffman Coding arXiv:0806.4899



Truncated Huffman coding

- Truncated Huffman coding představuje kompromis mezi složitostí kodéru a redundancí výsledného kódování.
- Princip:
 - Nahradit 2^c symbolů s nejmenší pravděpodobností virtuálním symbolem x .
 - Pomocí Huffmanova algoritmu zjistit kódová slova pro redukovanou sadu symbolů.
 - Kódové slovo přiřazené virtuálnímu symbolu x použít jako prefix pro 2^c symbolů.
- Příklad:
 - Uvažujme 10 symbolů abecedy, $P(a) = \{0.2, 0.14, 0.12, 0.12, 0.1, 0.1, 0.08, 0.07, 0.05, 0.02\}$, max. délka kódové značky 4 bity $\Rightarrow c = 2$. Určíme $P_x = 0.08 + 0.07 + 0.05 + 0.02 = 0.22$.
 - $H = 3.149$, $\langle f_{\text{truncHC}} \rangle = 3 \cdot (0.2 + 0.14 + 0.12 + 0.12 + 0.1 + 0.1) + 4 \cdot P_x = 3.22$, $R = \frac{3.22 - 3.149}{3.149} = 2\%$

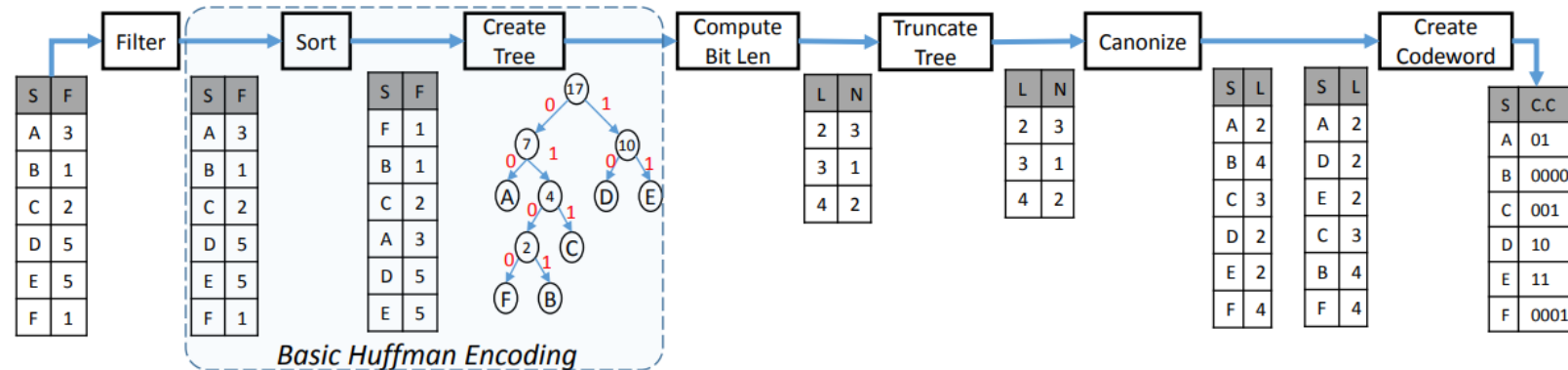


0	0.20	000
1	0.14	001
2	0.12	010
3	0.12	011
4	0.10	100
5	0.10	101
X	0.22	11xx

6	0.08	1100
7	0.07	1101
8	0.05	1110
9	0.02	1111

HW implementace reálného kodéru

■ Matai et. al Energy Efficient Canonical Huffman Encoding, 2014



- **Filter** module only passes symbols with non-zero frequencies
- **Create Tree** module builds the Huffman tree
- The **Compute Bit Len** module calculates the bit lengths of each codeword.
- The **Truncate Tree** module rebalances the Huffman tree when it is very tall and/or unbalanced. This improves decoder speed at the cost of a slight increase in encoding time.
- **Canonize** module creates two sorted lists. The first list contains symbols and frequencies sorted by symbol. The second list contains symbols and frequencies sorted by frequency. These lists are used for faster creation of the canonical Huffman codewords.
- The **Create Codeword** module creates uniquely decodable codewords.

Adaptivní Huffmanovo kódování

- Pokud není známo rozdělení pravděpodobností předem, Huffmanovo kódování se stává dvouprůchodovou metodou.
 - první průchod: nashromáždění statistik, bufferování dat, druhý průchod: konstrukce kódových slov
- *Adaptivní (dynamické) Huffmanovo kódování* – jednopřůchodová varianta, která nepotřebuje znát pravděpodobnosti předem, neboť se určují a aktualizují průběžně, jak přicházejí vstupní symboly
 - výhodné pokud lokální pravděpodobnosti v datech vykazující odlišné charakteristiky oproti globálním
 - vhodné pro multimédia, telekomunikaci, kde není možné použít dvouprůchodovou metodu
- Typicky se používá zrcadlená struktura kodéru a dekodéru

```
def AdaptiveHuffmanEncoder():  
    initHuffmanTree()  
    while not EOF:  
        c = getchar()  
        encode(c)  
        updateHuffmanTree(c)
```

```
def AdaptiveHuffmanDecoder():  
    initHuffmanTree()  
    while not EOF:  
        c = decode()  
        output(c)  
        updateHuffmanTree(c)
```

Adaptivní Huffmanovo kódování

■ Možnosti inicializace stromu (*initHuffmanTree*)

1. Naplnit strom všemi symboly abecedy a všem symbolům přiřadit četnost jedna
 - Výhody: netřeba řešit problém s prvním výskytem symbolu (každý symbol má vždy přiřazeno kódové slovo)
 - Nevýhody: pomalá adaptace (viz opakující se symbol A u abecedy s 256 symboly)
2. Začít s „prázdným“ stromem, do kterého se postupně budou přidávat symboly (viz FGK, Vitter)
 - Výhody: rychlá adaptace, menší redundance, výhodné zejména pokud je použito jen málo symbolů abecedy
 - Nevýhody: je třeba se vypořádat s prvním výskytem symbolu

■ Aktualizaci stromu je možné provádět (*updateHuffmanTree*)

1. naivně, tj. s každým načteným symbolem
 - Výhody: efektivní z pohledu redundance
 - Nevýhody: nepraktické z důvodu značné výpočetní náročnosti (každý symbol vyvolá aktualizaci sl. $O(n)$)
2. pravidelně, např. každých 1000 symbolů
 - Výhody: výrazně nižší výpočetní náročnost (1000 x)
 - Nevýhody: snížená kompresní účinnost
3. pokud jsou kódy nevyvážené (např. při změně pořadí symbolů z důvodů změny četnosti)
4. pomocí FGK (Faller, Gallagher, Knuth) algoritmu, který řeší problém efektivní aktualizace stromu
5. pomocí Vitterova algoritmu (efektivnější oproti FGK, vede na nižší redundanci)

Inicializace stromu – varianta s plně inicializovaným stromem

- Huffmanův strom je typicky inicializován pomocí dohodnutého mechanismu, který musí být znám kodéru i dekodéru.
- Máme-li abecedu N symbolů a každému přiřadíme počet výskytů roven jedné, pak na počátku získáme strom přiřazující každému symbolu binární kód délky $\lceil \log_2 N \rceil$ bitů (viz ukázky nejnižšího a nejvyššího Huffmanova stromu).
- Omezení: Lze aplikovat pouze pokud počet různých symbolů je předem znám.
 - Např. při kódování binárních dat víme, že ve vstupním toku se vyskytuje max. 256 různých hodnot.
- Nevýhoda: Adaptace může být relativně pomalá zejména u velkých abeced.
 - Aby se kódové slovo neustále opakujícího se symbolu zkrátilo z $\lceil \log_2 N \rceil$ bitů na 1 bit, musí se symbol na vstupu vyskytnout alespoň $N/2$ krát.
- Nevýhoda: Pokud se na vstupu vyskytuje jen malé množství různých symbolů, nutnost mít ve stromu všechny symboly zhoršuje kompresní účinnost
 - Všem značkám musí být přiřazen kód určité délky.

Inicializace stromu – varianta s plně inicializovaným stromem

- Příklad pro $N = |\Sigma| = 16, \Sigma = \{A, \dots, P\}$, vstupní tok $s = AB^n$
 - 1011,11111,110,110,10,10,10,10,0,0,0,... až od osmého výskytu B se začnou používat kódová slova délky 1 bit, $R=3 + 4+2 \times 2+4 \times 1=15$ bitů na prvních 8 symbolů

symbol	freq(a)	$f_{HC}(a)$
A	1	1011
B	1	1100
C	1	1010
D	1	0111
E	1	1101
F	1	1000
G	1	1001
H	1	1110
I	1	1111
J	1	0000
K	1	0001
L	1	0010
M	1	0011
N	1	0100
O	1	0101
P	1	0110

symbol	freq(a)	$f_{HC}(a)$
A	2	110
B	1	11111
C	1	0000
D	1	0001
E	1	0010
F	1	0011
G	1	0100
H	1	0101
I	1	0110
J	1	0111
K	1	1000
L	1	1001
M	1	1010
N	1	1011
O	1	1110
P	1	11110

symbol	freq(a)	$f_{HC}(a)$
A	2	101
B	2-3	110
C	1	11101
D	1	11110
E	1	11111
F	1	0000
G	1	0001
H	1	0010
I	1	0011
J	1	0100
K	1	0101
L	1	0110
M	1	0111
N	1	1000
O	1	1001
P	1	11100

symbol	freq(a)	$f_{HC}(a)$
A	2	011
B	4-7	10
C	1	11000
D	1	11001
E	1	11010
F	1	11011
G	1	11100
H	1	11101
I	1	11110
J	1	11111
K	1	0000
L	1	0001
M	1	0010
N	1	0011
O	1	0100
P	1	0101

symbol	freq(a)	$f_{HC}(a)$
A	2	1001
B	8+	0
C	1	11101
D	1	11100
E	1	11111
F	1	11110
G	1	10111
H	1	10110
I	1	11011
J	1	11000
K	1	11010
L	1	11001
M	1	10101
N	1	10100
O	1	10001
P	1	10000

Inicializace stromu – varianta s jedním uzlem

- Abychom co nejvíce minimalizovali režii adaptivní implementace, je výhodnější začít s prázdným stromem a postupně do něj vkládat nově příchozí symboly.
- Problém: symbol, který ještě není ve stromu, nemá přiřazeno kódové slovo
 - jak dekodér rozliší, zda-li se jedná o nový symbol nebo kódové slovo symbolu v listovém uzlu stromu?
- NYT symbol (0-node) se speciálním významem
 - Pokud symbol není ve stromu, tak se vyšle kódové slovo odpovídající NYT (not yet transmitted) symbolu následované hodnotou nového symbolu kódovanou fixním počtem bitů. Uzel NYT se rozdělí na nový uzel NYT (levý potomek) a list s nově načteným symbolem (pravý potomek).
 - Pokud symbolu NYT přiřadíme nízkou a konstantní váhu (četnost výskytu), pak bude vytlačován častěji se vyskytujícími symboly a režie bude malá. Kódové slovo NYT symbolu se postupně prodlužuje s každou aktualizací stromu.
- Počáteční Huffmanův strom obsahuje dva symboly – NYT a první symbol vstupu
 - Hodnota prvního symbol je zapsána přímo do výstupního toku a je kódována fixním počtem bitů
 - Pro kódování nových symbolů a prvního symbolu je možné použít tzv. *zfázovaný binární kód* (angl. *phased-in binary code*)

Vsuvka: Phased-in binary code

- Motivace: Předpokládejme abecedu $\Sigma = \{a_1, \dots, a_N\}$. Máme-li N symbolů, kódová slova kódu fixní délky musí mít $\log_2 N$ bitů. Je-li $2^m \leq N < 2^{m+1}$, pak $\log_2 N$ není celé číslo a v případě běžného binárního kódování musíme použít $\lceil \log_2 N \rceil$ bitů a ne všechny kombinace využijeme.
- Kódování *phased-in binary code* přiřazuje hodnotám kódovou značku délky buď m nebo $m + 1$ bitů. Cílem je získat kódování jehož průměrná délka se blíží $\log_2 N$.
 - Určíme hodnotu celého čísla m tak, aby $2^m \leq N < 2^{m+1}$ a určíme $r = N - 2^m$.
 - Prvních $2r$ symbolů se kóduje pomocí $(m + 1)$ -bitového čísla v rozsahu hodnot 0 až $2r - 1$.
 - Zbývajících $N - 2r$ symbolů se kóduje pomocí m -bitového čísla tak, že kód a_k odpovídá $k - r - 1$.
- Phased-in binární kódování je jednoznačně dekódovatelné
 - Platí, že $\sum_{i=1}^N 2^{-L_i} = 2r \cdot 2^{-(m+1)} + (N - 2r) \cdot 2^{-m} = r2^{-m} + (N - 2r)2^{-m} = 2^{-m}(N - r) = 2^{-m}(N - N + 2^m) = 1$, tzn. je splněna Kraft-McMillanova nerovnost.
 - Dekodér nejprve načte m bitů a pokud je hodnota menší než r , načte ještě chybějící bit. Načtená hodnota odpovídá po převodu a přičtení jedničky hodnotě symbolu.
- Příklad:
 - Nechť $N=26$, pak $m=4$ a $r=10$.
 - Symbol a_1 se zakóduje jako 00000, a_2 jako 00001, a_{20} jako 10011, a_{21} jako 1010 a a_{26} pomocí 1111
 - $L_{avg} = \frac{1}{26}(20 \cdot 5 + 6 \cdot 4) = 4.77$, $\log_2 N = 4.70$

Inicializace stromu – varianta s jedním uzlem

- Příklad pro $N = |\Sigma| = 16, \Sigma = \{A, \dots, P\}$, tzn. $m = 4, r = 0$, vstupní tok $s = AB^n$
 - Strom obsahuje symbol NYT, ke kterému se po přečtení prvního symbolu přidá symbol A. Na výstup je zapsán přímo 4-bitový binární kód symbolu.

symbol	freq(a)	$f_{HC}(a)$
NYT	0	0
A	1	1

- V dalším kroku se přidá uzel B a postupně zvyšující se váha modifikuje Huffmanův strom

symbol	freq(a)	$f_{HC}(a)$
NYT	0	10
B	1	11
A	1	0

symbol	freq(a)	$f_{HC}(a)$
NYT	0	00
B	2	1
A	1	01

symbol	freq(a)	$f_{HC}(a)$
NYT	0	00
B	3+	1
A	1	01

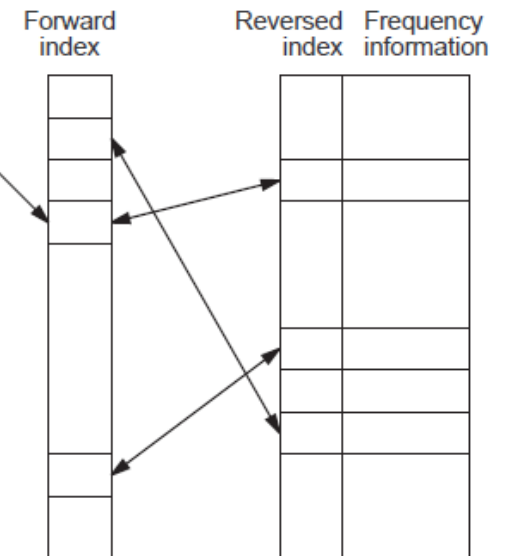
- Výstup: 0000 ('A'), 0, 0001 ('B'), 01, 0, 0, 0, 0, 0, Již od druhého výskytu B se začnou používat kódová slova délky 1 bit (rychlejší adaptace), $R = 4 + 5 + 1 = 10$ bitů na první 3 symboly (nižší redundance)

Aktualizace stromu – při změně pořadí

- Zjistit, zda-li jsou kódové značky nevyvážené je možné například pomocí detekce změn vzájemného pořadí jednotlivých symbolů.
- Detekovat změnu pořadí lze efektivně pokud budeme udržovat dva seznamy o délce odpovídající počtu symbolů abecedy každý
 - *forward index (FI)*, který udržuje pro každý symbol ukazatel (index) do druhého seznamu
 - druhý seznam obsahuje pro každý symbol dvojici (symbol, počet výskytů symbolu)

■ Algoritmus:

- Začínáme lineárním mapováním s inicializací všech počtů na hodnotu 1.
- Pro načtený symbol zjistíme z FI index příslušné položky a zvýšíme počet výskytů
- Porovnáme, jestli počet výskytů je vyšší než počet výskytů předcházející položky a pokud ano, provedeme výměnu s předchozí položkou a aktualizujeme forward index (prohodíme hodnoty na odpovídajících dvou indexech). Výměnu provádíme tak dlouho, dokud nezískáme seřazený seznam (viz Bubble sort, symbol probublává směrem nahoru).
- Pokud došlo k alespoň jedné výměně, provedeme přepočítání Huffmanova stromu.
- Experimentální výsledky na textových souborech Calgary Corpus:
 - Huffmanův strom se přepočítá pro 2-5% načtených symbolů



Porovnání účinnosti jednotlivých přístupů

Uvažujme abecedu 256-ti různých ASCII symbolů, vstupní řetězec $s = (\text{abcdef})^{100}$

- Entropie:

- $H(S) = \log_2 6 = 2.585$

- Statické Huffmanovo kódování:

- f_{CHC} : $a \rightarrow 111, b \rightarrow 100, c \rightarrow 110, d \rightarrow 00, e \rightarrow 101, f \rightarrow 01, \langle f_{CHC} \rangle = 2.67$ bitů na symbol, výstupní tok včetně hlavičky: 2.8 bitů na symbol

- Adaptivní Huffmanovo kódování:

Způsob inicializace stromu	Způsob aktualizace	bitů na symbol	Počet aktualizací
NYT + první symbol	Po každém symbolu (naivní)	3.36	600
256 symbolů, uniformní	Po každém symbolu (naivní)	3.87	600
256 symbolů, uniformní	Pokud počet je mocnina 2	4.09	36
256 symbolů, uniformní	Při změně pořadí	7.01	6
256 symbolů, uniformní	Při změně pořadí nebo mocnina 2	4.06	36

- Algoritmus FGK (NYT): 2.88 bitů na symbol

Porovnání účinnosti jednotlivých přístupů

Uvažujme abecedu 256-ti různých ASCII symbolů, vstup 'paper1' (Calgary Corpus, 52 kB)

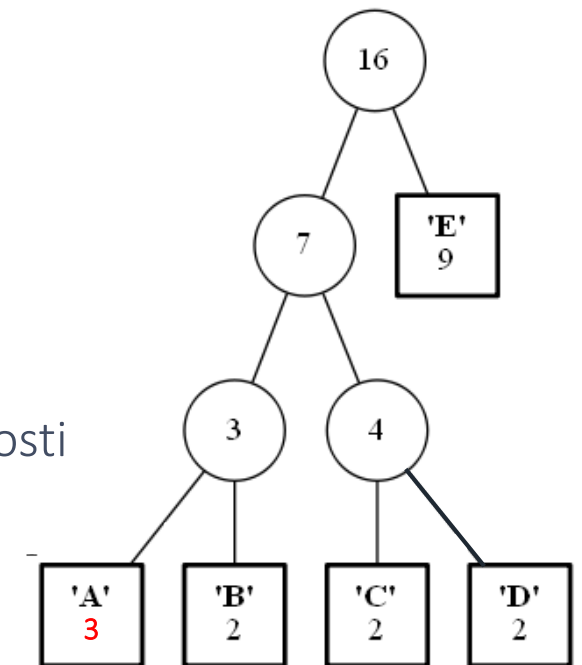
- Entropie:
 - $H(S) = 4.983$
- Statické Huffmanovo kódování:
 - $\langle f_{CHC} \rangle = 5.017$ bitů na symbol, výstupní tok včetně hlavičky: 5.034 bitů na symbol
- Adaptivní Huffmanovo kódování:

Způsob inicializace stromu	Způsob aktualizace	bitů na symbol	Počet aktualizací
NYT + první symbol	Po každém symbolu	5.0350	53161
256 symbolů, uniformní	Po každém symbolu	5.0516	53161
256 symbolů, uniformní	Pokud počet je mocnina 2	5.0558	613
256 symbolů, uniformní	Při změně pořadí	5.0520	2038
256 symbolů, uniformní	Při změně pořadí nebo mocnina 2	5.0518	2606

– Algoritmus FGK (NYT): 5.0327 bitů na symbol

Adaptivní Huffmanovo kódování

- Algoritmus FGK (Faller, Gallager, Knuth) využívá tzv. *sourozenecké vlastnosti* (angl. sibling property) Huffmanova stromu
 - Algoritmus funguje na principu vkládání symbolů do stromu a v případě, že se detekuje porušení sourozenecké vlastnosti, provede se přeuspořádání stromu tak, aby byla sourozenecká vlastnost zachována.
 - Gallager dokázal, že binární prefixový kód je Huffmanovým kódem právě tehdy, když má odpovídající kódovací strom sourozeneckou vlastnost.
- Binární strom má *sourozeneckou vlastnost* jestliže
 1. každý uzel (s výjimkou kořenového) má sourozence a
 2. uzly lze seřadit (podle ohodnocení daného uzlu) do monotónní posloupnosti tak, že má každý uzel v posloupnosti za souseda svého sourozence.



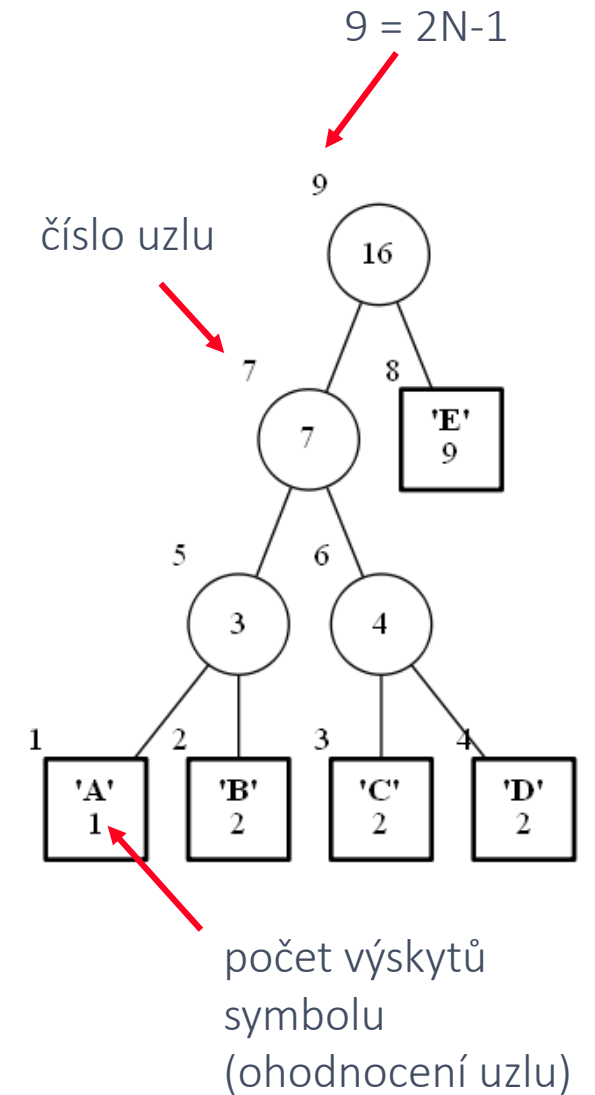
Adaptivní Huffmanovo kódování

■ Říkáme, že Huffmanův strom je *lexikograficky uspořádaný*, pokud

1. hrana vedoucí k potomku s menší četností má přiřazenu hodnotu 0,
2. všechny uzly v hloubce d mají menší četnosti výskytu (ohodnocení) než uzly v hloubce $d - 1$ a
3. uzly v stejné hloubce jsou uspořádány do neklesající posloupnosti.

■ Důsledek sourozenecké vlastnosti Huffmanova stromu:

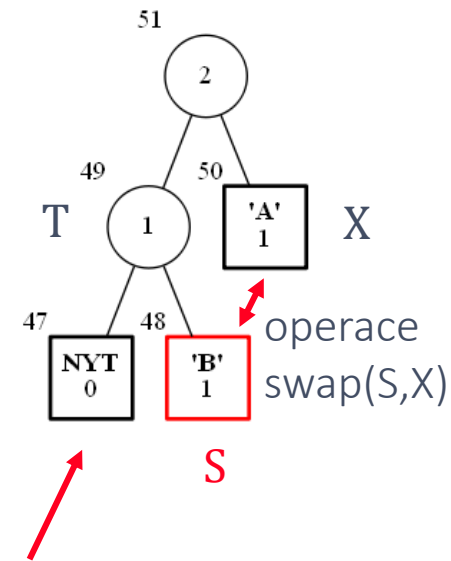
- Všem uzlům je možné přiřadit číslo od 1 do $2N - 1$ (N odpovídá velikosti abecedy, hodnota $2N - 1$ je přiřazena kořenovému uzlu) tak, že každá dvojice uzlů s čísly $2i - 1$ a $2i$ ($i = 1 \dots N - 1$) má společného rodiče a pro četnosti přiřazené uzlům platí $freq_1 \leq freq_2 \leq \dots \leq freq_{2N-1}$.
- Strom splňující podmínku sourozenecké vlastnosti je lexikograficky uspořádaný strom.



Algoritmus FGK – aktualizace stromu

- Požadavek, aby byly frekvence na všech úrovních seřazeny zleva doprava je umělý. Principiálně není nutný, ale zjednoduší se proces aktualizace stromu.
- Algoritmus FGK navržený Gallagerem pracuje s plně inicializovaným stromem, avšak je možné jej zobecnit na variantu s jedním NYT uzlem (viz dále).
- Aktualizace stromu probíhá rekurzivně a začíná v listovém uzlu načteného symbolu. Nechť S je listový uzel odpovídající načtenému symbolu.

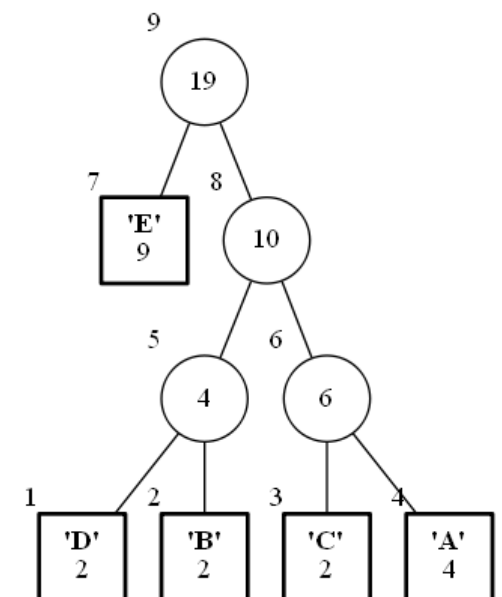
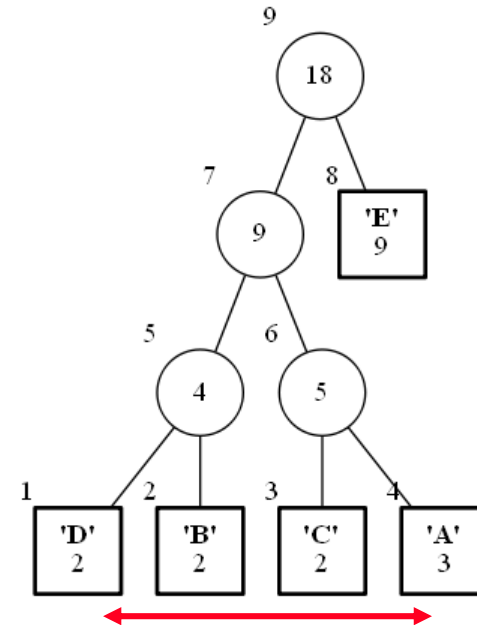
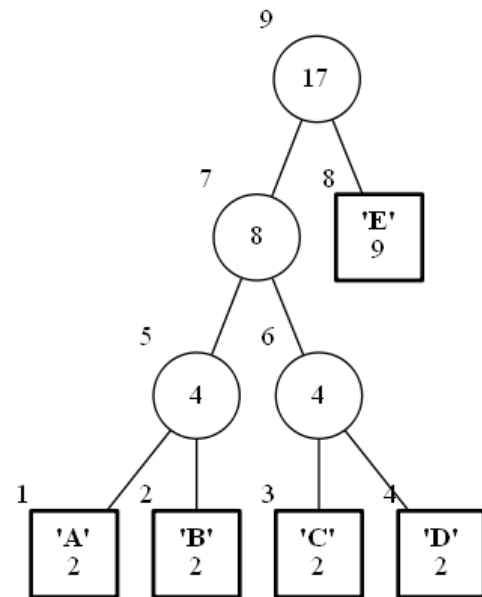
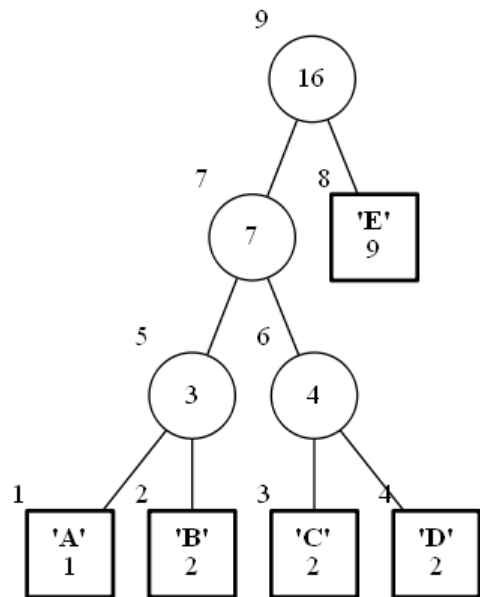
1. Nechť T je bezprostřední následovník uzlu S . Pokud $freq(S) < freq(T)$, pak uzly jsou uspořádané. V opačném případě se prohodí uzel S nejpravějším uzlem X (operace *swap*), pro který platí, že $freq(X) = freq(S)$. Uzel X může být ve stejné či nižší hloubce (viz ukázka). Nikdy nesmí dojít k prohození se svým rodičem.
2. Čítač počtu výskytů uzlu S se zvýší na $freq(S) = freq(S) + 1$
3. Nyní se přejde do nového rodiče uzlu S (tj. původní rodič X provedla-li se výměna), tj. $S = root(S)$ a kroky 1 až 3 se opakují pro rodičovský uzel. Cyklus končí jakmile narazíme na kořenový uzel.



všechny ostatní symboly
(Not Yet Transmitted)

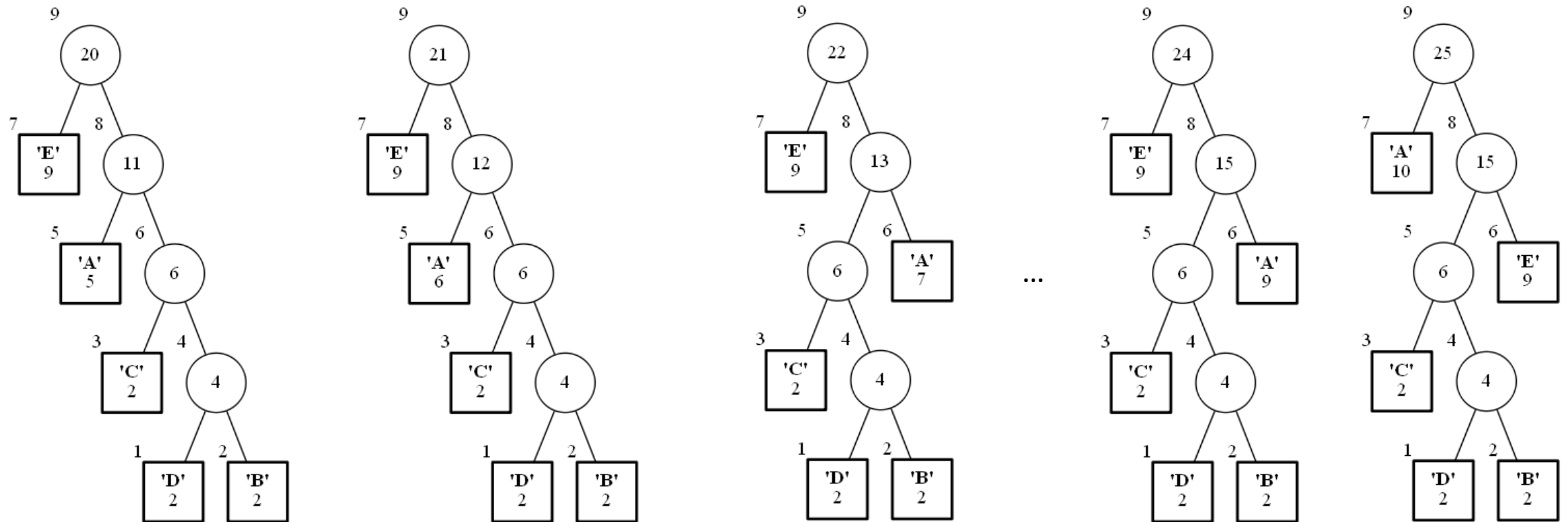
Algoritmus FGK – aktualizace stromu

- Princip aktualizace stromu s poč. stavem 1,2,2,2,3,4,7,9,16, $\Sigma = \{A, B, C, D, E\}$, pro AAAA



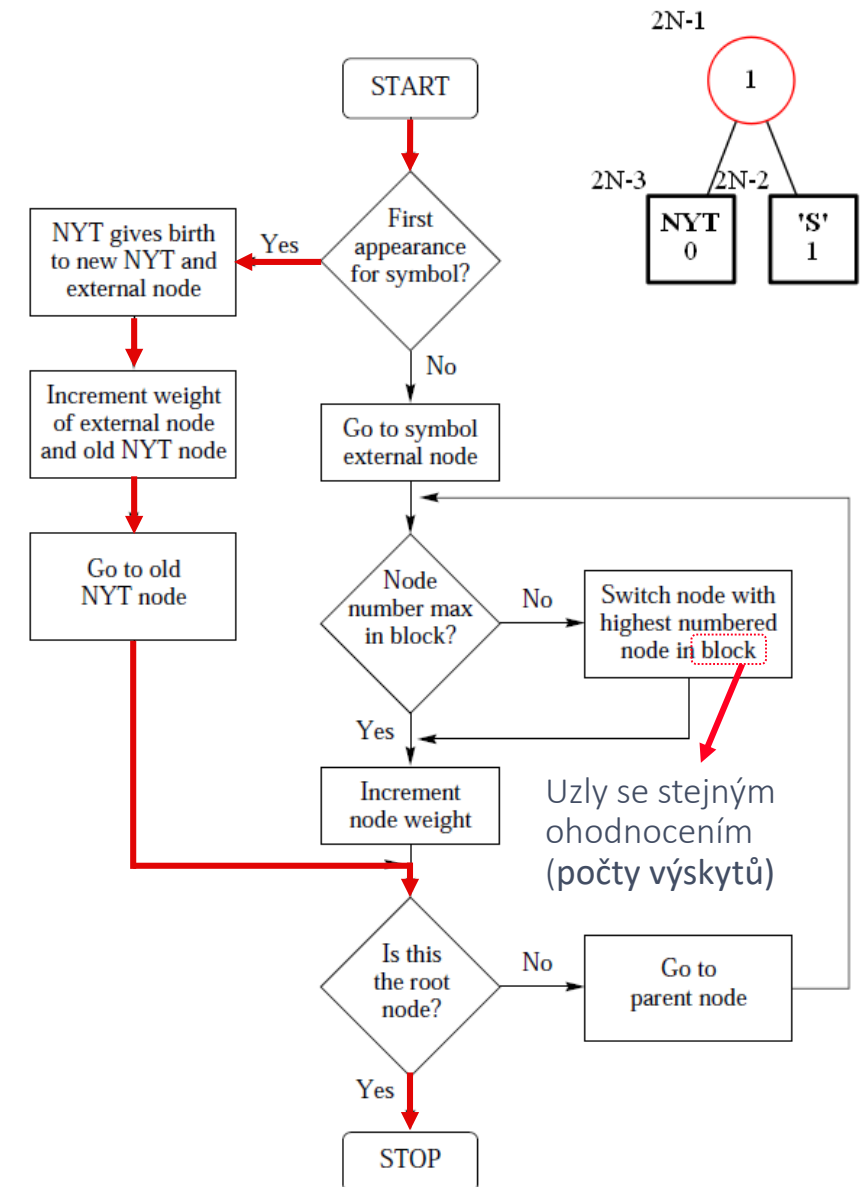
Algoritmus FGK – aktualizace stromu

- Princip aktualizace stromu s poč. stavem 1,2,2,2,3,4,7,9,16 pro AAAAAAAAAA



Algoritmus FGK – aktualizace stromu s jedním uzlem

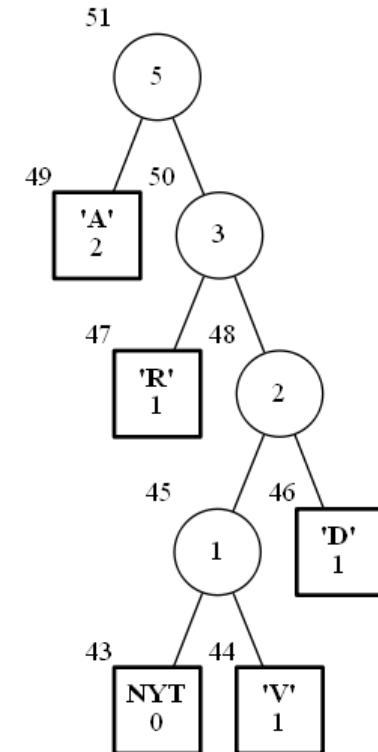
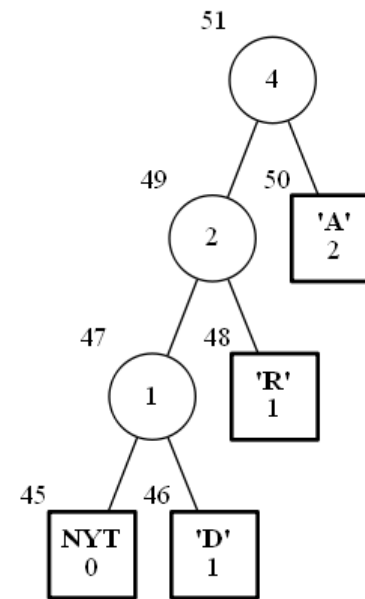
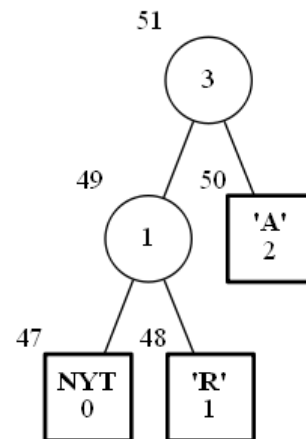
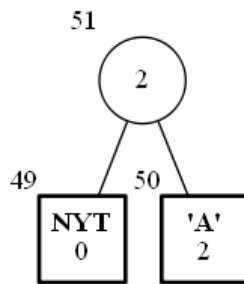
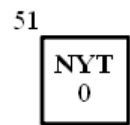
- Pořadí uzlů je zachováno pokud přiřadíme hodnoty čísel následovně:
 - NYT uzel bude mít vždy přiřazenu nejnižší hodnotu, kořenový uzel nejvyšší hodnotu
 - Počáteční hodnota čísla NYT uzlu je rovna $2N - 1$.
- Pokud se načte symbol, který není ve stromu, pak
 - Z listového NYT uzlu majícího číslo i se stane uzel vnitřní se dvěma potomky, kterým se přiřadí čísla $i - 2$ a $i - 1$. Všechny tři uzly budou mít nulové ohodnocení.
 - Potomek $i - 2$ odpovídá novému listovému NYT uzlu, potomek $i - 1$ listovému uzlu načteného symbolu.



Algoritmus FGK – aktualizace stromu s jedním uzlem

- Příklad: uvažujme abecedu $N=26$ symbolů a vstupní řetězec $s = \mathbf{aardv}$

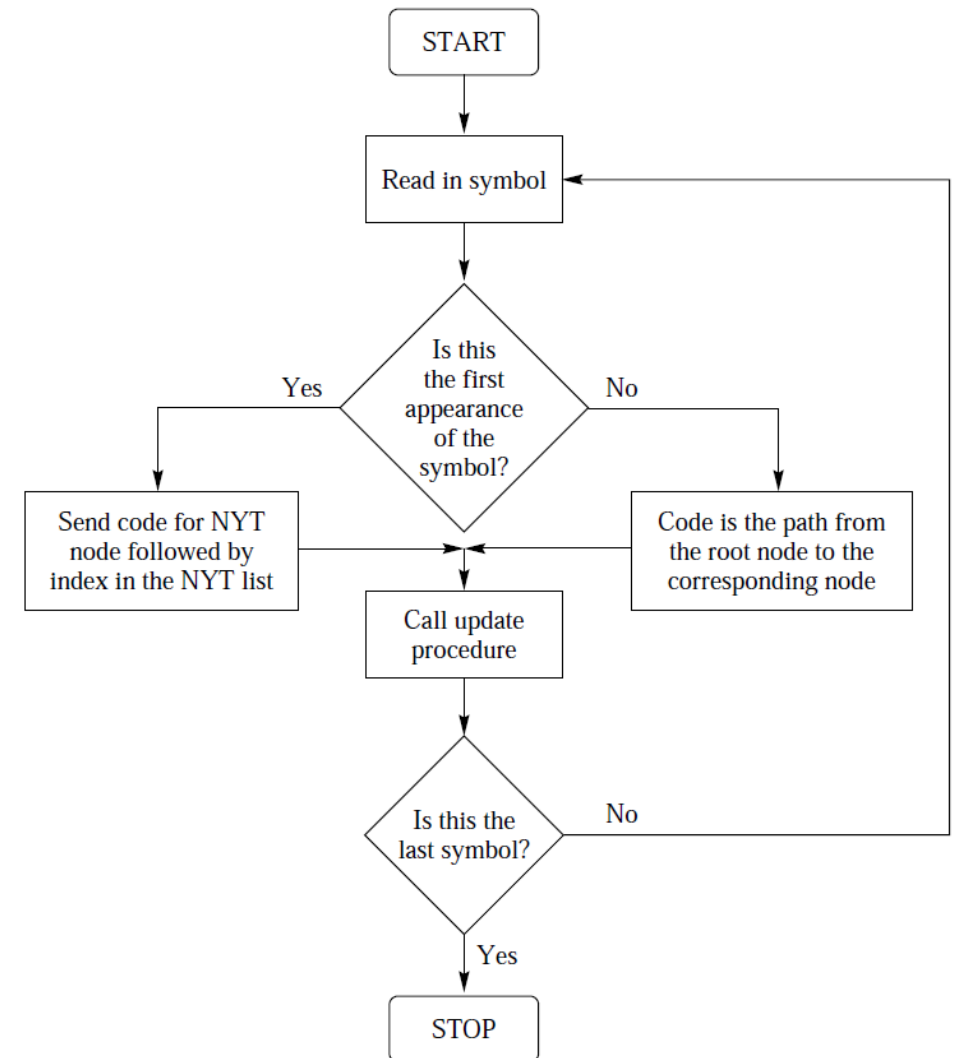
strom obsahuje $2N-1=51$ uzlů, výstup: 0 00000 ('a') 1 0 10001 ('r') 00 00011 ('d') 000 1011 ('v')



Algoritmus FGK – kodér

Algoritmus kodéru

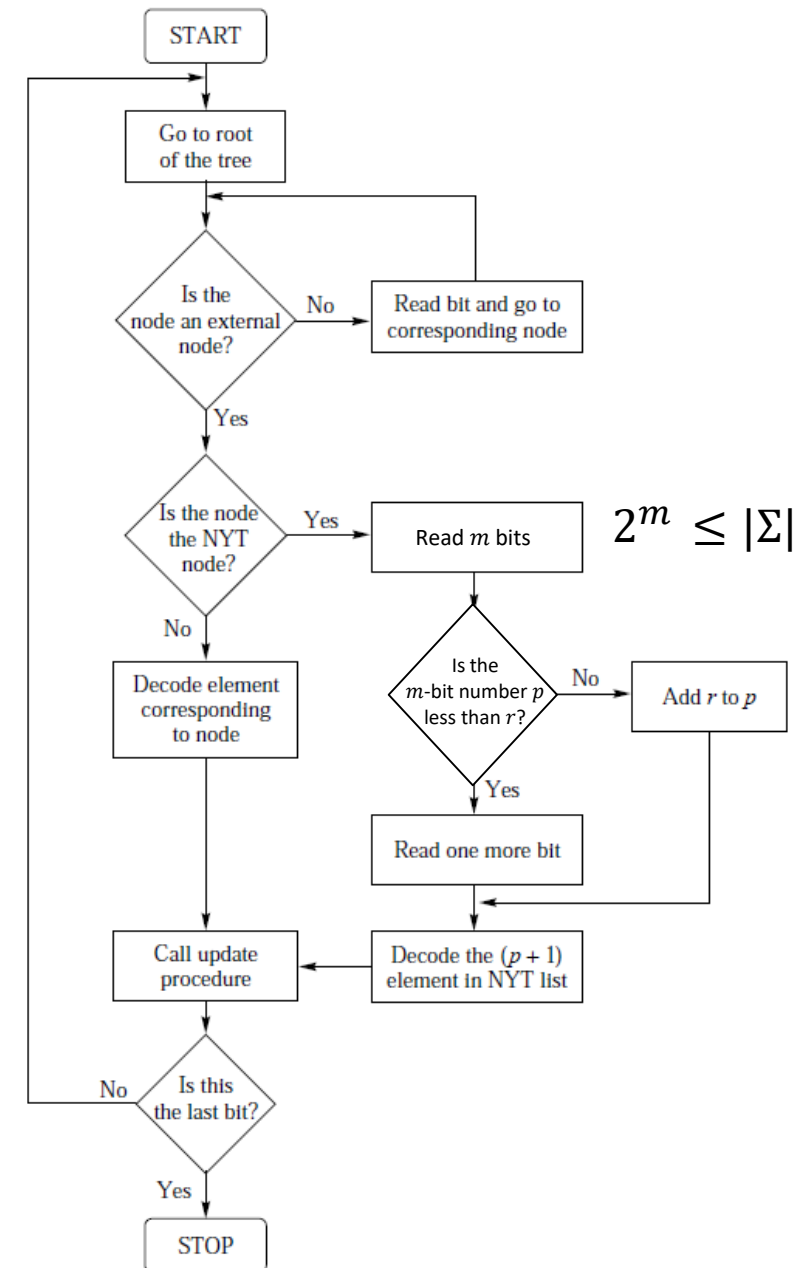
- Kodér začíná s jediným uzlem (NYT), kterému je přiřazena váha 0
- Následně je načten symbol ze vstupu.
 - Pokud se jedná o nový symbol, na výstup je zapsáno kódové slovo odpovídající NYT následované indexem symbolu (phased code)
 - Pokud se jedná o symbol ve stromu, pak se na výstup запиše kódové slovo, které získáme průchodem od kořene směrem k listu
- Poté je volán algoritmus pro aktualizaci stromu, který
 - Zajistí rozdvojení NYT v případě nového symbolu
 - Inkrementaci uzlů a modifikaci pořadí uzlů tak, aby byla zaručena sourozenecká vlastnost



Algoritmus FGK – dekodér

Algoritmus dekodéru

- Dekodér začíná také s jediným uzlem (NYT), kterému je přiřazena váha 0
- Následně je postupně vstup čten bit po bitu a dle hodnot bitů se pohybujeme ve stromu (0 – levá větev, 1 – pravá větev) tak dlouho, než narazíme na listový uzel
 - Pokud se jedná o NYT uzel, načteme index nového symbolu.
 - Pokud se jedná o uzel symbolu, pak se na výstup zapíše odpovídající symbol
- Poté je volán algoritmus pro aktualizaci stromu, který
 - Zajistí rozdvojení NYT v případě načtení nového symbolu
 - Inkrementaci uzlů a modifikaci pořadí uzlů tak, aby byla zaručena sourozenecká vlastnost

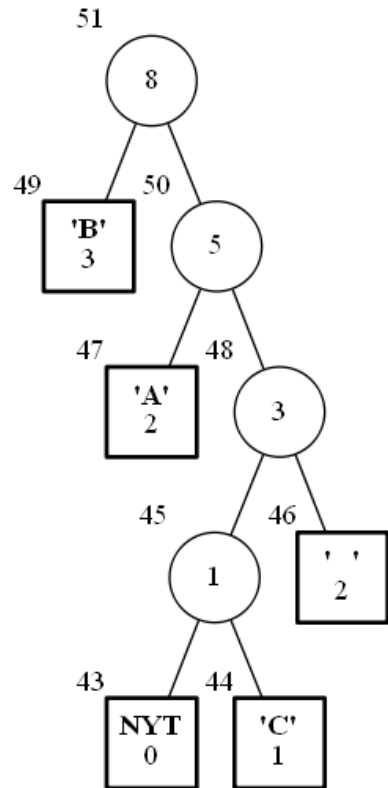


Algoritmus V – Vitter a efektivita FGK

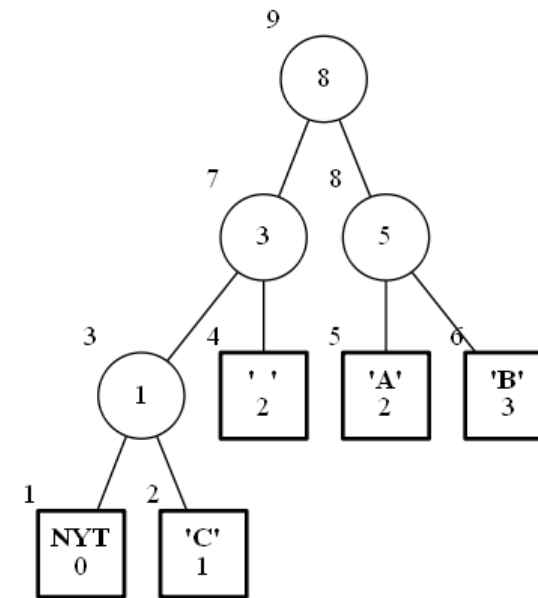
- Vitter (1986) navrhl vylepšení FGK algoritmu označované jako algoritmus V, který dosahuje vyšší účinnosti komprese.
 - Vitter ukázal, že algoritmus produkuje v nejhorším případě méně než $S + t$ bitů, zatímco FGK může produkovat až $2S + t$ bitů, kde S je průměrný počet bitů na symbol, kterého je možné dosáhnout pomocí statického Huffmanova kódování a t je délka vstupu.
 - Později bylo dokázáno, že *FGK v nejhorším případě dává dvojnásobný počet bitů na symbol v porovnání se semistatickou verzí Huffmanova kódování* (Milidiu, Laber and Pessoa, 2000)
- Princip: zavedení tvrdší podmínky na pořadí uzlů s cílem udržovat strom s minimálními možnými délkami kódových značek a co nejmenší maximální délkou kódové značky.
 - všechny listové uzly váhy w musí předcházet všechny interní uzly váhy w
- Při výměně uzlů se mohou vyměnit pouze uzly stejné váhy a stejného typu (tj. pouze listové nebo pouze interní).
 - Průchod stromem v pořadí uzlů produkuje alternující sekvenci listových a vnitřních uzlů
- Algoritmus používá implicitní uspořádání uzlů
 - uzly jsou očíslovány postupně podle hloubky a v každé hloubce zleva doprava.
- Detailní popis algoritmu viz Algorithm 673: Dynamic Huffman coding
 - <https://dl.acm.org/citation.cfm?id=214390>

Porovnání efektivity FGK a V algoritmu

- Tvar kódového stromu pro vstupní řetězec „aa bbb c“
 - Strom získaný V algoritmem má nejen nižší výšku (3 vs. 4) ale i celkovou délku značek (12 vs. 14).
 - V případě, že další symbol bude c, na výstup se zapíše 001 (vs. 1101 v případě FGK)



Strom získaný FGK algoritmem



Strom získaný algoritmem V

Algoritmus V – aktualizace stromu

- Pokud se načte symbol, který není ve stromu, pak
 - Z listového NYT uzlu se stane uzel vnitřní se dvěma potomky – levý NYT uzel, pravý symbol. Oba s nulovým ohodnocením. Přesuneme se na původní NYT uzel a poznačíme si, že je nutné aktualizovat také pravého potomka.
- Pokud pro symbol existuje uzel ve stromu
 - Provedeme výměnu uzlu s nejpravějším uzlem bloku (tzv. leader). Blokem rozumíme uzly stejného typu se stejným ohodnocením.
 - Pokud se jedná o sourozence NYT uzlu, poznačíme si, že je nutné jej aktualizovat a přesuneme se na rodiče uzlu.
- Aktualizujeme strom
 - Postupně voláme operaci SlideAndIncrement pro rodiče dokud nenarazíme na kořen stromu.
 - Pro označený uzel zavoláme operaci SlideAndIncrement, která provede finální uspořádání.

```
def Update_TreeVITTER(symbol):

    leafToIncrement = NULL;
    p = pointer to symbol's node;

    if ( p == NULL ): /* a new symbol */
        Create two leaf children of the 0-node,
        such that the right child is the new
        symbol node and the left child is
        the new 0-node;
        p = parent of the symbol node;
        leafToIncrement = the right child of p;
    else:
        Swap p in the tree with the leader of its block;
        if ( p is the sibling of the 0-node ):
            leafToIncrement = p;
            p = parent of p;

    while ( p != root of the tree ):
        /* if possible, advance p to the next block. */
        p = SlideAndIncrement( p )

    if ( leafToIncrement != NULL ):
        SlideAndIncrement(leafToIncrement);
```

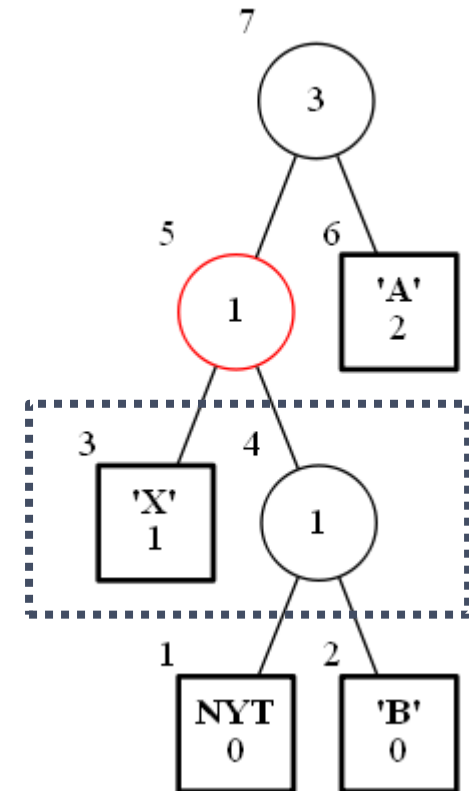
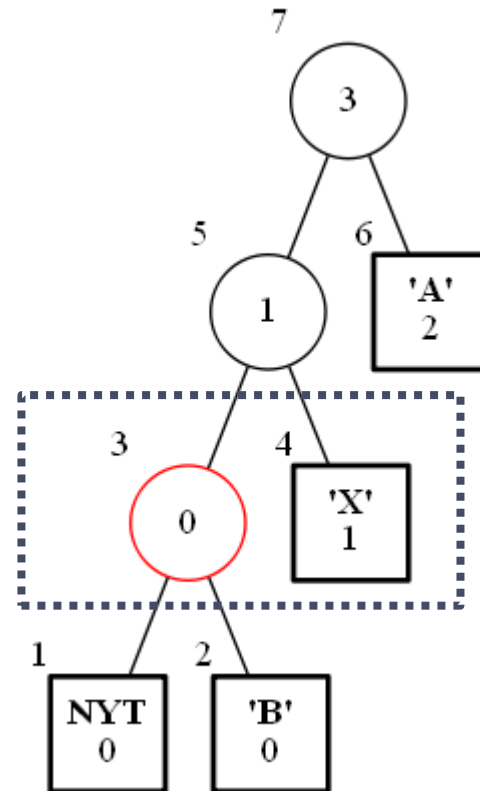
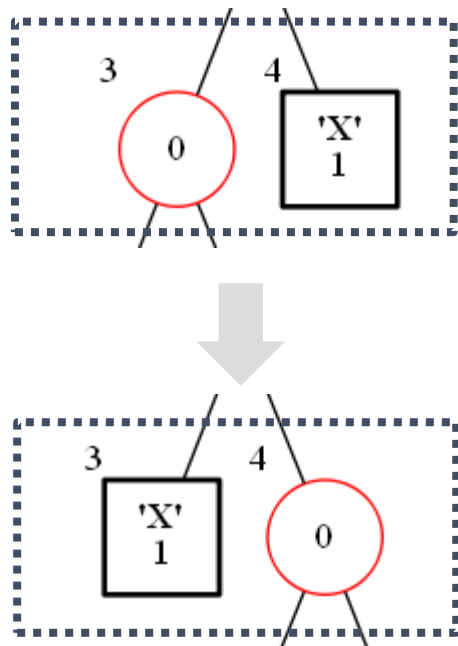
Algoritmus V – operace *SlideAndIncrement*

- Operace *SlideAndIncrement* je založena na cyklickém posuvu uzlů tak, aby se aktivní uzel dostal na správnou pozici a bylo zachováno požadované uspořádání.
- Pro vnitřní uzel s ohodnocením w se provede cyklický posuv uzlů tak, aby se uzel dostal za listové uzly váhy $w + 1$
- Pro listový uzel s ohodnocením w se provede cyklický posuv uzlů tak, aby se uzel dostal před uzly s ohodnocením w
- Následně se zvýší ohodnocení uzlu a přechází se na rodičovský uzel původního uzlu (vnitřní uzel) nebo rodičovský uzel uzlu po cyklickém posuvu.

```
def slideAndIncrement ( accept node p ):  
    fp = parent of p;  
    w = p's weight;  
  
    if ( p is an internal node ):  
        Slide p in the tree higher than  
        the leaf nodes of weight w+1;  
    else:  
        Slide p in the tree higher than  
        the nodes of weight w;  
  
    p's weight = p's weight + 1;  
  
    if (p is an internal node):  
        p = fp  
    else:  
        p = new parent of p  
  
    return p
```

Princip SlideAndIncrement

- Pro vnitřní uzel s ohodnocením w se provede cyklický posuv uzlů vlevo tak, aby se uzel dostal za listové uzly s ohodnocením $w + 1$. Následně se inkrementuje ohodnocení uzlu a přechází se na rodičovský uzel původního uzlu.
 - Uzel 3: $w = 0$

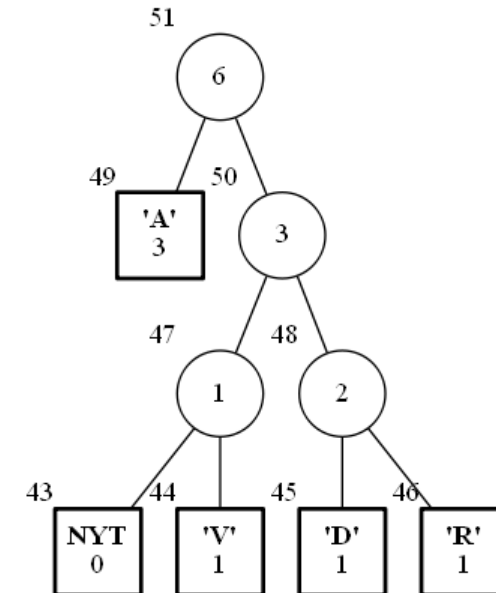
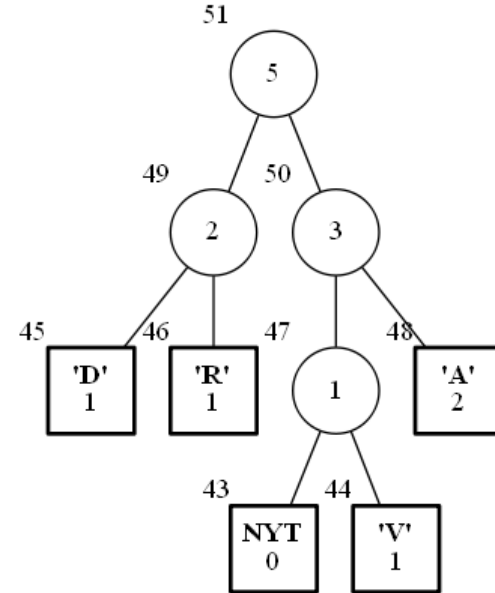
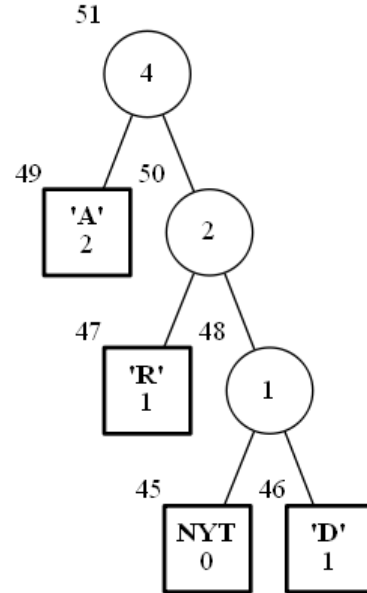
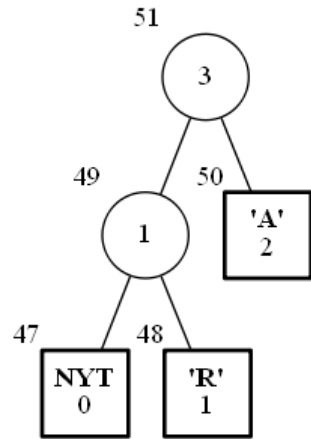


Algoritmus V – aktualizace stromu s jedním uzlem

- Příklad: uvažujme abecedu $N=26$ symbolů a vstupní řetězec $s = \text{aardvark}$

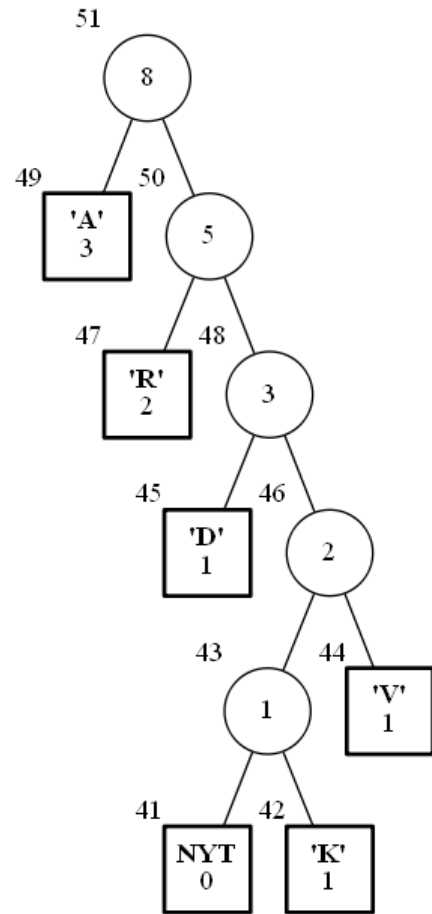
strom obsahuje $2N-1=51$ uzlů, výstup: 0 00000 ('a') 1 0 10001 ('r') 00 00011 ('d') 110 01011 ('v') 11 01

Stromy pro první tři aktualizace jsou totožné s FGK

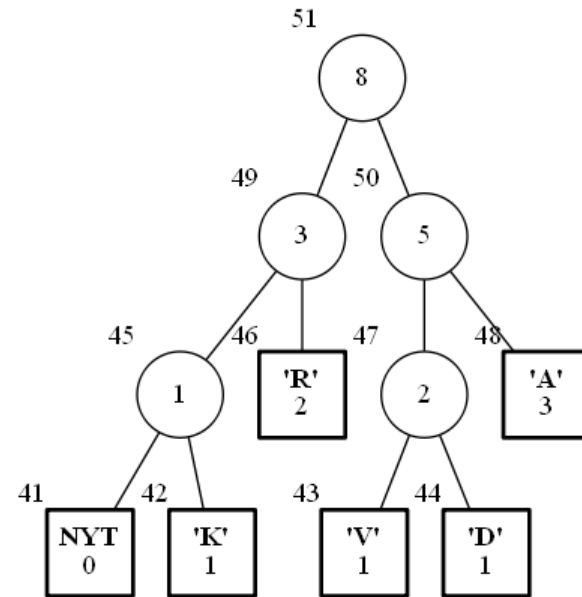


Porovnání tvaru stromu pro jednotlivé algoritmy

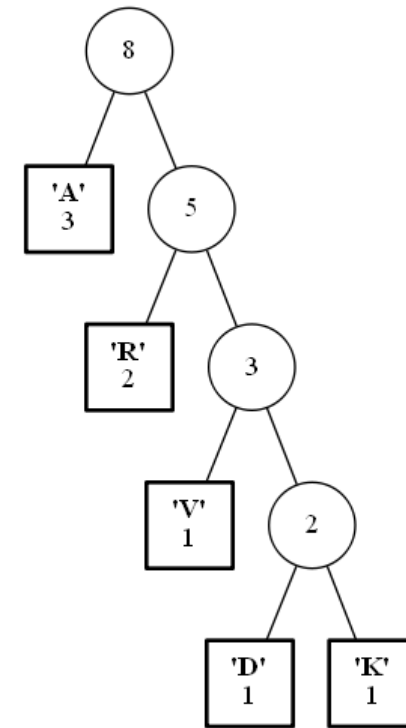
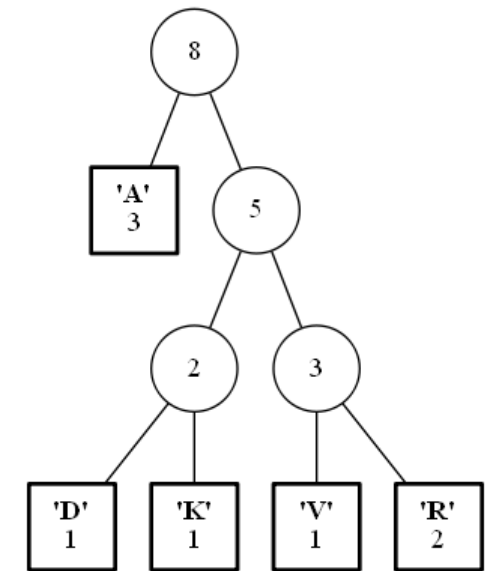
■ Řetězec aardvark



Adaptivní algoritmus FGK



Adaptivní algoritmus Vitter

Semistatický
Huffmanův algoritmusSemistatický
Huffmanův algoritmus
(min variance)

Praktické problémy Huffmanova kódování

■ Přeplnění čítače a změna měřítko

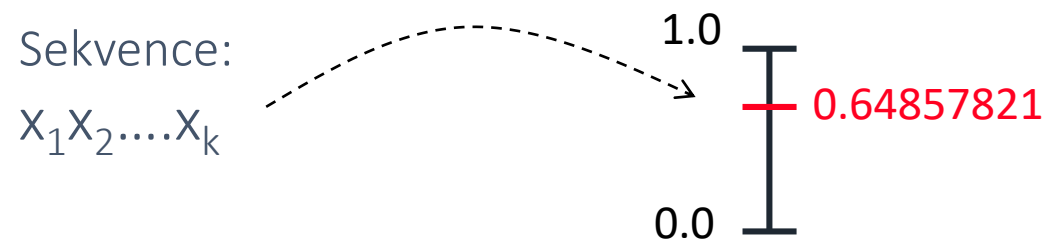
- Hodnoty četností se uchovávají v proměnných s pevným počtem bitů a může dojít k přetečení. Pole s 16-bitovými položkami bez znaménka může střídat počty až do 65 535.
- Možností je sledovat ohodnocení kořene při každé inkrementaci a když dosáhne maximální hodnoty, provést změnu měřítko (rescale) všech čítačů frekvence dělením 2 (celočíslné dělení).
- Jinou možností je použít pole 32-bitových hodnot, kde k přetečení nebude docházet tak často.
- Po změně měřítko mají nové symboly větší vliv než staré (počítané před změnou měřítko). Ukazuje se, že to je příznivé, protože ze zkušenosti se ví, že pravděpodobnost výskytu symbolu závisí více na bezprostředně předcházejícím symbolu, než na symbolu, který se vyskytl v daleké minulosti.

■ Přeplnění kódu

- Dalším problémem je přeplnění kódu v důsledku dlouhých kódových značek. Kodér prochází stromem a střídá jednotlivé bity v nějaké proměnné pevného počtu bitů, neboť jsou potřeba na výstup zapsat v opačném pořadí. Pokud je počet bitů nedostatečný, může dojít k vyčerpání předtím, než narazí na listový uzel.
- Jednou možností je použít vázaný seznam, to však může být pomalé.

Aritmetické kódování

- Aritmetické kódování (Elias, 1960; Abramson 1963) je nejefektivnější kódování, které umí dosáhnout meze entropie.
 - první praktické realizace: Rissanen (76), Pasco (76), Rubin (79), Moffat (98) a Witten (87).
- Princip: problém přidělování kódů celočíselné délky jednotlivým symbolům se eliminuje tak, že se přidělí jeden kód (velmi dlouhý) celému vstupnímu toku $x_1x_2\dots x_k$.
 - Vstupní řetězec chápeme jako tok nekonečné délky
 - Dovoluje použití neceločíselného počtu bitů na symbol $|f(x_1x_2\dots x_k)| = \alpha k$, kde $\alpha \in \mathbb{R}$
 - Výstupem je hodnota v intervalu $[0, 1]$
- Vlastnosti mapování
 - Musí záviset na pravděpodobnosti výskytů symbolů
 - Musí být sestaveno v průběhu kódování (není nutné a priori znát všechna mapování)



Aritmetické kódování

■ Princip:

- Metoda startuje s určitým intervalem – např. $[0,1)$, čte vstupní řetězec symbol po symbolu a používá pravděpodobnosti výskytu jednotlivých symbolů k zužování intervalu.
- Pro každý symbol x_i vstupní sekvence se rozdělí interval podle pravděpodobností výskytů symbolů abecedy $p_1, p_2, \dots, p_n$ a následně se již pracuje pouze s tím podintervalem, kterému přísluší vstup x_i
- Kódem je číslo z intervalu, který jsme obdrželi po k krocích.

■ Důsledky

- Výstupem je hodnota, která jednoznačně identifikuje konečný interval $\Rightarrow$ jednoznačná dekódovatelnost
- Specifikace užšího intervalu vyžaduje další desetinná místa $\Rightarrow$ délka binární reprezentace hodnoty, kterou algoritmus konstruuje průběžně roste.

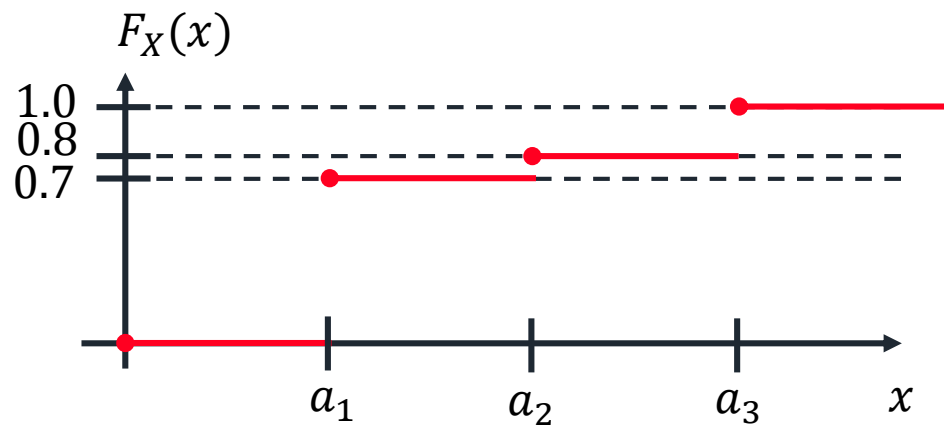
■ Pravděpodobnosti výskytu symbolů

- mohou být během kódování konstantní (statická varianta aritmetického kódování)
- se mohou měnit (adaptivní varianta aritmetického kódování)

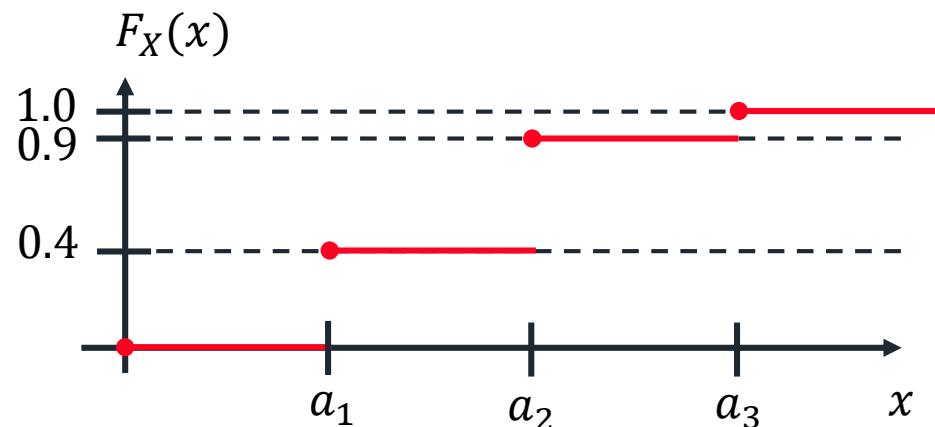
Aritmetické kódování

- Kumulativní distribuční funkce (CDF) diskrétní náhodné proměnné $X \in \{a_1, a_2, a_3\}$:

$$F_X(x) = P(X \leq x) = \sum_{x_i \leq x} P(X = x_i)$$



$$P(X = a_1) = 0.7, P(X = a_2) = 0.1, P(X = a_3) = 0.2$$



$$P(X = a_1) = 0.4, P(X = a_2) = 0.5, P(X = a_3) = 0.1$$

0.8 = b0.110011001100110...

0.7 = b0.101100110011001...

0.9 = b0.111001100110011...

0.4 = b0.011001100110011...



Aktualizace intervalu

- Pro každý nový symbol je nutné
 - určit dolní (l) a horní (u) mez nového intervalu a
 - rozdělit interval dle pravděpodobnosti výskytu jednotlivých symbolů

- Inicializace:

$$l^{(0)} = 0 \qquad u^{(0)} = 1$$

- Aktualizace:

$$\begin{aligned}
 l^{(n)} &= l^{(n-1)} + [u^{(n-1)} - l^{(n-1)}] F_X(\text{prev}(x_n)) \\
 u^{(n)} &= \underbrace{l^{(n-1)} + [u^{(n-1)} - l^{(n-1)}]}_{\text{z předchozího intervalu}} \underbrace{F_X(x_n)}_{\text{z pravd. modelu}}
 \end{aligned}$$

$$S = x_1 x_2 x_3 \cdots x_k, \quad x_i \in \Sigma$$

$$\Sigma = \{a_1, a_2, \dots, a_k\}$$

$$\text{prev}(a_i) = \begin{cases} \perp & \text{pokud } i=1 \\ a_{i-1} & \text{jinak} \end{cases}$$

$$F_X(\perp) = 0$$



Aktualizace intervalu – vlastnosti

- Jaká je nejmenší hodnota, kterou může mít $l^{(n)}$?

$$l^{(n)} = l^{(n-1)} + \underbrace{[u^{(n-1)} - l^{(n-1)}]}_{>0} \underbrace{F_X(\text{prev}(x_n))}_{\geq 0} \Rightarrow l^{(n)} \geq l^{(n-1)}$$

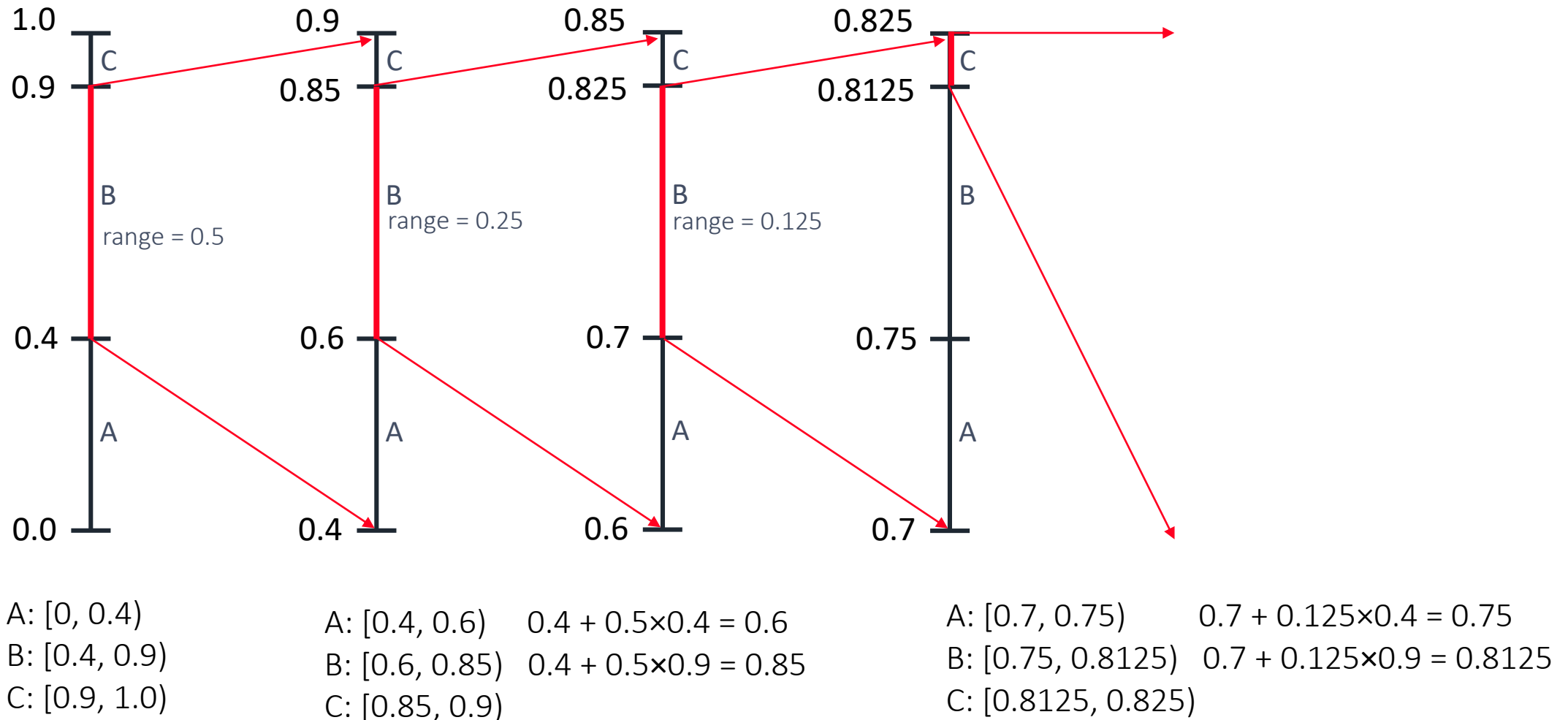
- Jaká je největší hodnota, kterou může mít $u^{(n)}$?

$$u^{(n)} = \cancel{l^{(n-1)}} + \underbrace{[u^{(n-1)} - \cancel{l^{(n-1)}}]}_{\leq 1} \underbrace{\cancel{F_X(x_n)}}_{\leq 1} \Rightarrow u^{(n)} \leq u^{(n-1)}$$

- Pro aktualizaci intervalu platí $[l^{(n)}, u^{(n)}] \subseteq [l^{(n-1)}, u^{(n-1)}]$, což je důležitá podmínka pro zajištění jednoznačné dekódovatelnosti
 - Rozdělení intervalu na základě znalosti F_X umožní jednoznačné dekódování

Aritmetické kódování – příklad

- Nechť $p(A)=0.4$, $p(B)=0.5$, $p(C)=0.1$ Mějme zakódovat vstupní sekvenci: BBBC...



Aritmetické kódování – příklad

- Mějme zakódovat vstupní sekvenci: BBBC

x	$p(x)$	Opt. # bitů $\log_2(1/p)$	Huffmanovo kódování
A	0,4	1,3219	10
B	0,5	1,0000	0
C	0,1	3,3219	11

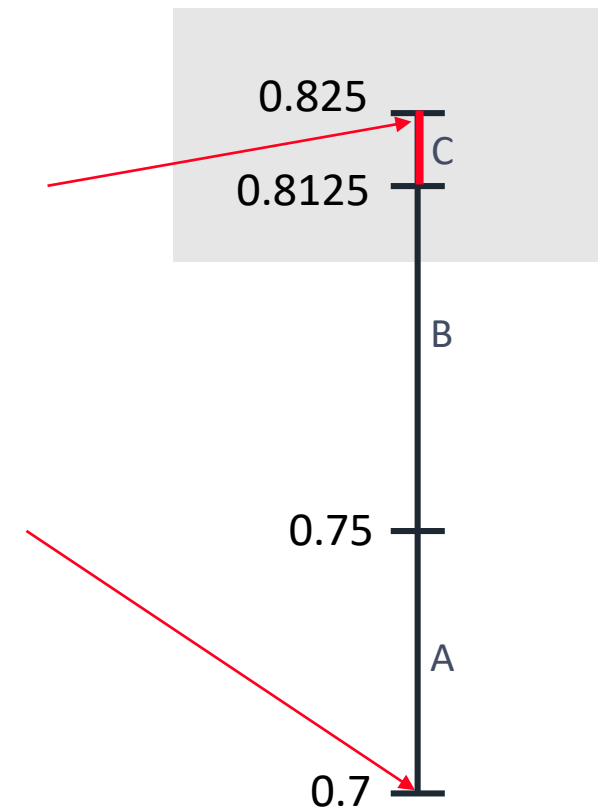
- Abychom odeslali BBBC, stačí zakódovat libovolnou z hodnot v ležící ve vypočteném intervalu, tj. $0.8125 \leq v < 0.825$

a	binární reprezentace
0,8125	0,1101
0,8250	0,110100 <u>1100</u> ...

4 bity pro aritmetické kódování

vs

5 bitů pro Huffmanovo kódování



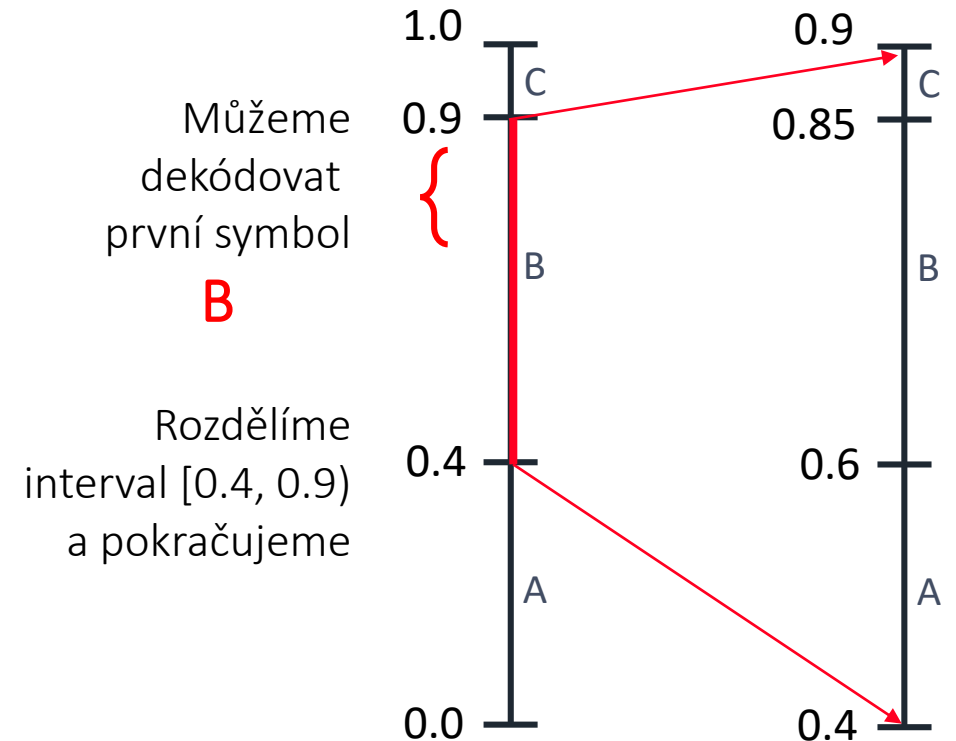
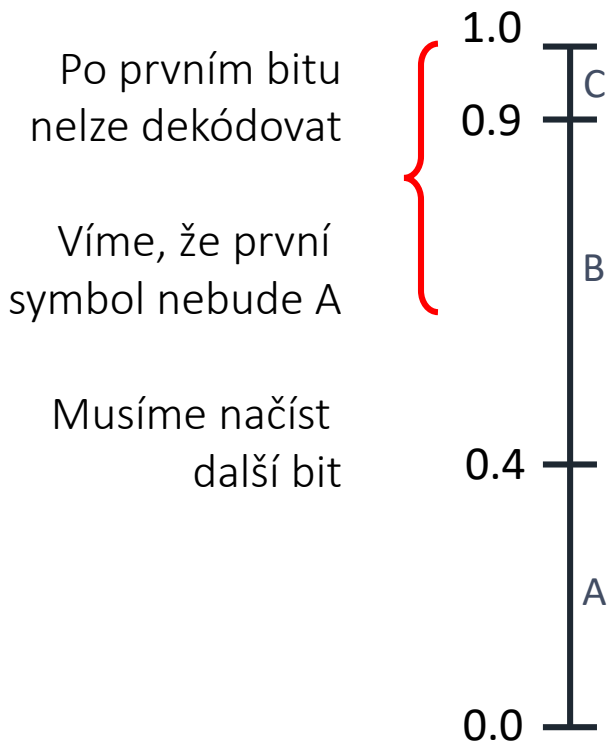
Dekódování zprávy

■ Dekódování zprávy 1101

$$1 \rightarrow \begin{cases} 0,11111111 \dots = 1 \\ 0,10000000 \dots = 0,5 \end{cases}$$

$$11 \rightarrow \begin{cases} 0,11111111 \dots = 1 \\ 0,11000000 \dots = 0,75 \end{cases}$$

$$110 \rightarrow \begin{cases} 0,11011111 \dots = 0,875 \\ 0,11000000 \dots = 0,75 \end{cases}$$



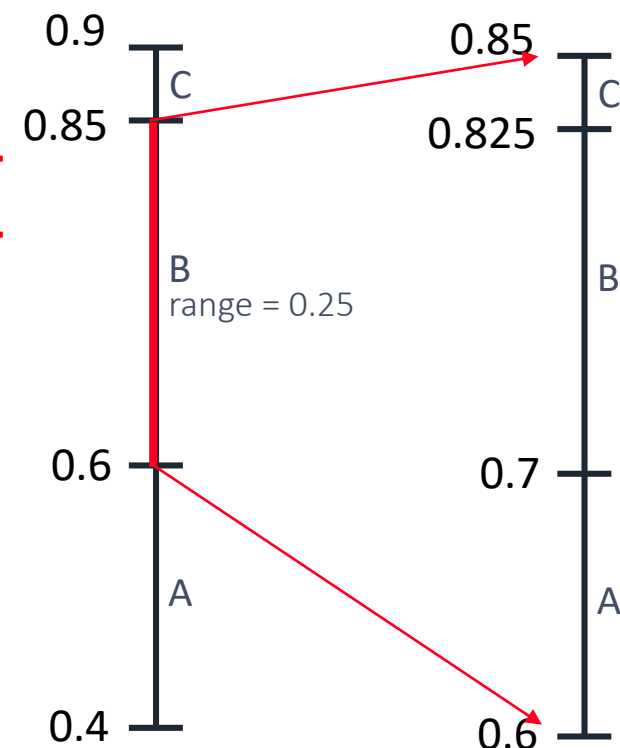
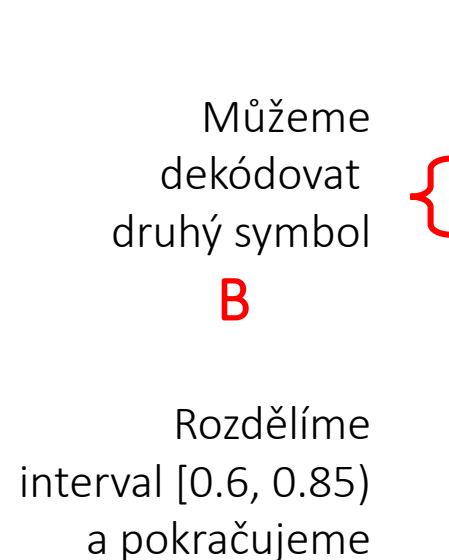
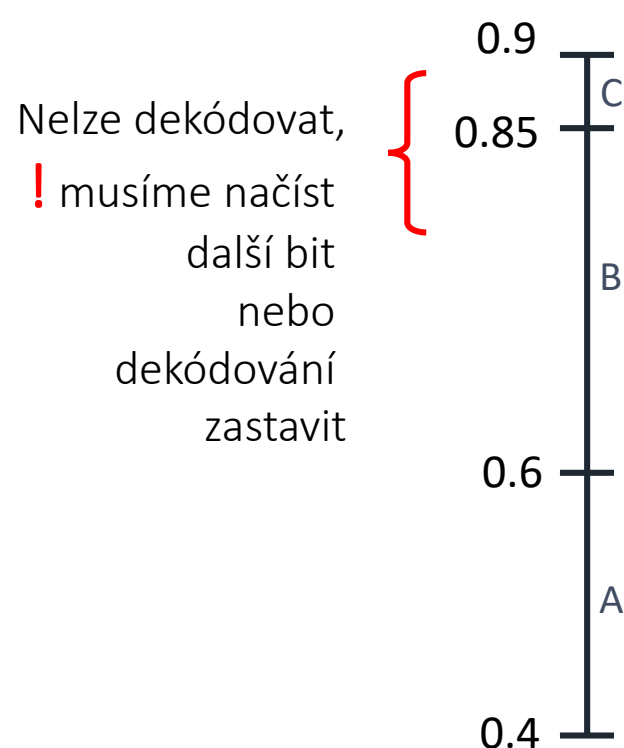
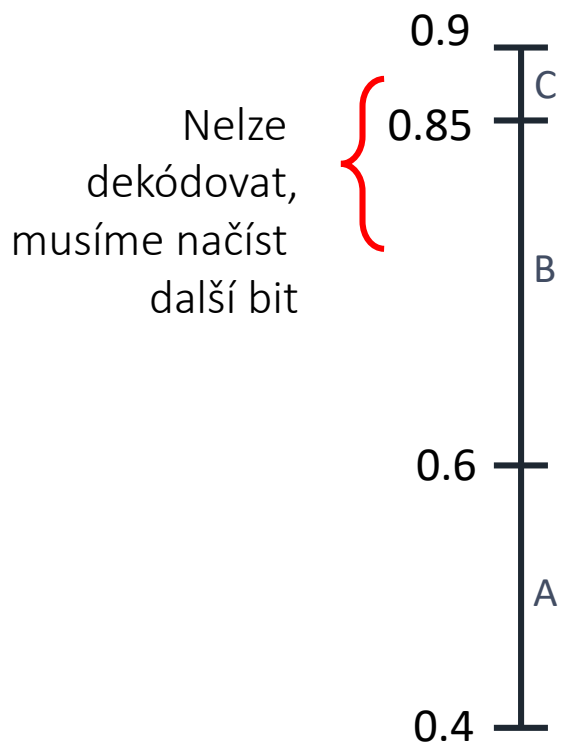
Dekódování zprávy

■ Dekódování zprávy 1101

$$110 \rightarrow \begin{cases} 0,110111111 \dots = 0,875 \\ 0,110000000 \dots = 0,75 \end{cases}$$

$$1101 \rightarrow \begin{cases} 0,110111111 \dots = 0,875 \\ 0,110100000 \dots = 0,8125 \end{cases}$$

$$11010 \rightarrow \begin{cases} 0,110101111 \dots = 0,84375 \\ 0,110100000 \dots = 0,8125 \end{cases}$$



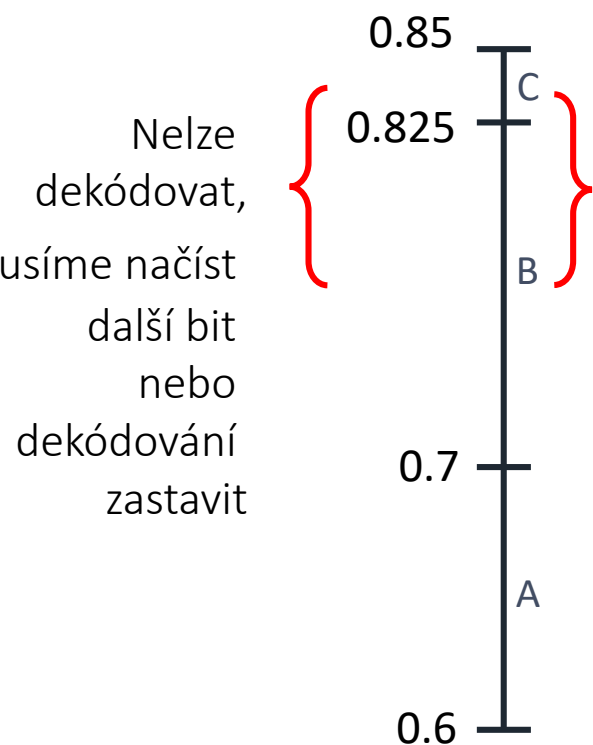
Dekódování zprávy

Dekódování zprávy 1101

$$11010 \rightarrow \begin{cases} 0,110101111 \dots = 0,84375 \\ 0,110100000 \dots = 0,8125 \end{cases}$$

$$1101000 \rightarrow \begin{cases} 0,11010001111 \dots = 0,8203125 \\ 0,11010000000 \dots = 0,8125 \end{cases}$$

$$110100 \rightarrow \begin{cases} 0,1101001111 \dots = 0,828125 \\ 0,1101000000 \dots = 0,8125 \end{cases}$$

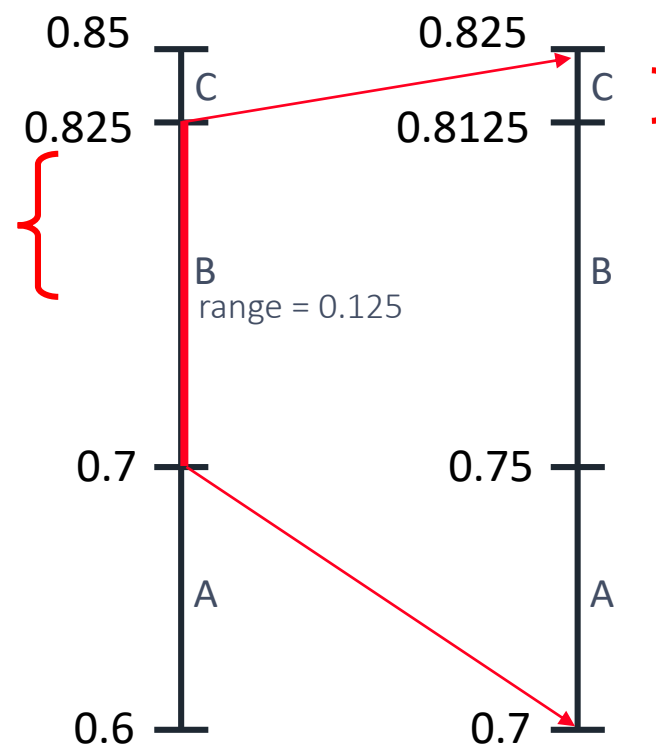


Nelze dekodovat

Můžeme dekodovat třetí symbol

B

Rozdělíme interval $[0.7, 0.825)$ a pokračujeme



Můžeme dekodovat čtvrtý symbol

C

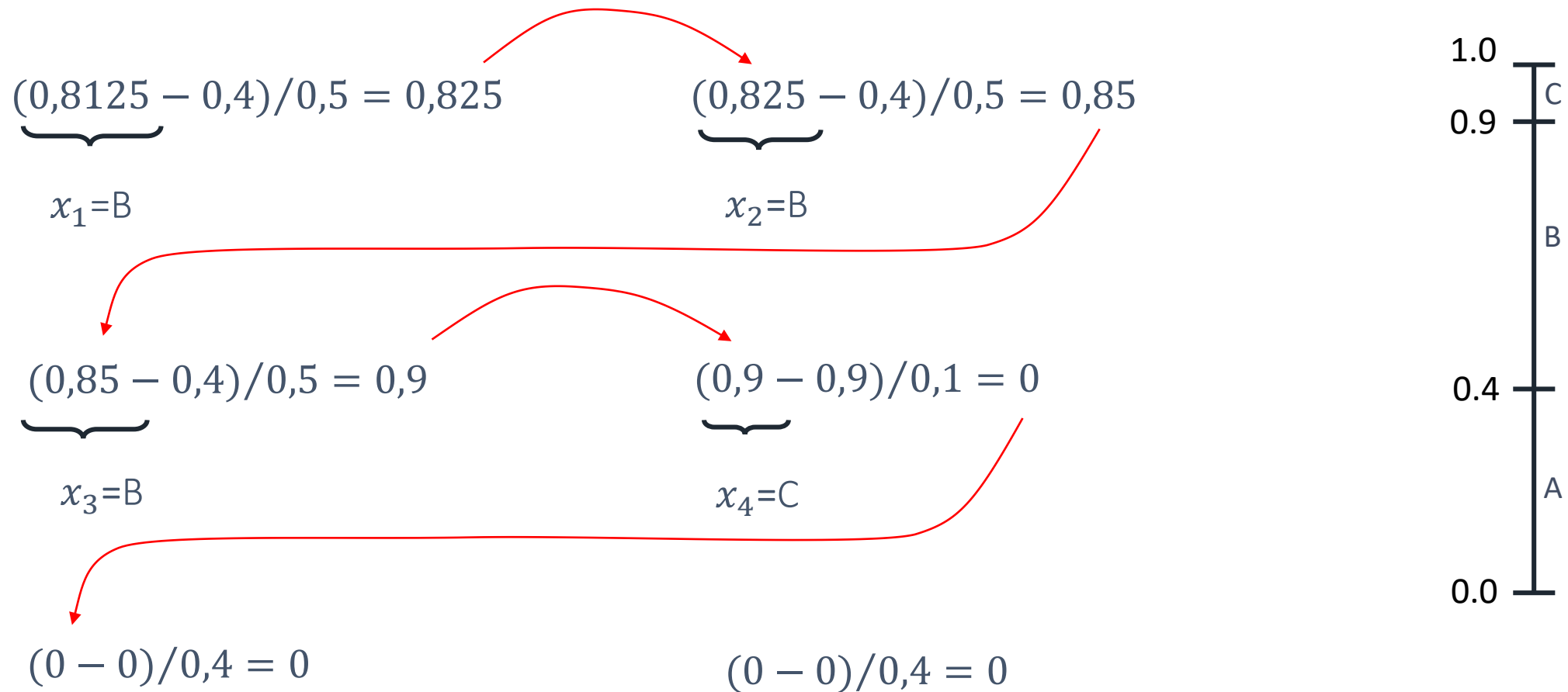
Rozdělíme interval $[0.8125, 0.825)$ a pokračujeme nebo skončíme ?

Alternativní způsob dekódování

- Alternativně je možné využít vztahu

$$code = (code - low) / range \quad low = F_X(prev(x_n)) \quad range = F_X(x_n) - F_X(prev(x_n))$$

- Příklad: Dekódování zprávy 1101 => 13/16



Reprezentace intervalu

■ Problémy

- Binární reprezentace desetinných čísel může vést na nekonečný počet bitů. Pro daný interval $[l, u)$ potřebujeme získat hodnotu na co nejmenší počet bitů
 - Např. $1/3 = 0.3\bar{3} = 0.0101010101\overline{01}$, nebo $2/3 = 0.6\bar{6} = 0.1010101\overline{0}$
- Práce s desetinnými čísly by vyžadovala mít k dispozici aritmetiku s nekonečnou přesností
- Operace dělení je z výpočetního hlediska velmi drahá operace

■ Praktická řešení

- QM kódér (binární reprezentace, násobení a dělení nahrazeno posuvy), použito v JPEG
- Intervalové kódování

Naivní implementace

■ Struktura kodéru:

- inicializace l,u
- zpracování symbolu (update l,u)
- získání výsledné hodnoty

■ Pro stanovení binární reprezentace lze využít Lemma:

- $\forall l, s \geq 0$ taková, že $l + s < 1$ platí, že zaokrouhlením hodnoty $l + s/2$ na prvních $-\lfloor \log_2 s \rfloor + 1$ bitů získáme hodnotu v rozsahu $[l, l + s)$; přičemž platí následující rovnost:

$$-\lfloor \log_2 s \rfloor + 1 = \lfloor \log_2 2/s \rfloor$$

```
> encoded_msg = ArithmeticCoder(frequency_table=
    {'a':4, 'b':5, 'c':1}).\
    encodestring("bbbc")
> print(encoded_msg)
11010001
```

```
class ArithmeticCoder:
    def __init__(self, frequency_table=None, code=None):
        self.l, self.u = Decimal(0.0), Decimal(1.0) #current interval bounds
        if frequency_table: self._updfreqtab(frequency_table)
        if code: self.code = Decimal(int(value,2))/Decimal(2**len(code))

    def _updfreqtab(self, frequency_table):
        """Calculate CDF"""
        ffrom = Decimal(0.0)
        self.cumintervals = {a:(ffrom, ffrom:=ffrom+b) for a,b in
                               frequency_table.items()}
        self.scale = Decimal(1.0)/ffrom

    def encode(self, symbol, frequency_table=None):
        """Encode one symbol"""
        if frequency_table: self._updfreqtab(frequency_table)
        range = (self.u - self.l)*self.scale
        self.l, self.u = self.l + range*self.cumintervals[symbol][0],
                          self.l + range*self.cumintervals[symbol][1]

    def finish(self):
        """Return binary string"""
        s = self.u - self.l
        bits = int(ceil(log(2/s,2)))
        s = (self.l + s/2) * (1<<bits)
        bb = bin(int(s))[2:]
        while len(bb)<bits: bb = '0' + bb
        return bb

    def encodestring(self, string):
        for ch in string:
            self.encode(ch)
        return self.finish()
```

Naivní implementace

■ Struktura dekodéru:

- inicializace (načtení kódu, CDF)
- dekódování symbolu ležícím v intervalu daným kódem a update kódu
- dekódujeme dokud nezískáme řetězec požadované délky

```
> decoded_msg = ArithmeticCoder(frequency_table=
    {'a':4, 'b':5, 'c':1}).\
    decodestring('11010001', 4)
> print(decoded_msg)
"bbbc"
```

```
class ArithmeticCoder:
    def __init__(self, frequency_table=None, code=None):
        self.l, self.u = Decimal(0.0), Decimal(1.0)
        if frequency_table: self._updfreqtab(frequency_table)
        if code: self.code = Decimal(int(code,2))/Decimal(2**len(code))

    def _updfreqtab(self, frequency_table):
        """Calculate CDF"""
        ffrom = Decimal(0.0)
        self.cumintervals = {a:(ffrom, ffrom:=ffrom+b) for a,b in
                             frequency_table.items()}
        self.scale = Decimal(1.0)/ffrom

    def decode(self, frequency_table=None):
        """Decode one symbol"""
        if frequency_table: self._updfreqtab(frequency_table)
        code = self.code/self.scale
        for ch,(l,h) in self.cumintervals.items():
            if l <= code < h:
                self.code = (code - l)/(h-l)
                return ch
        return None

    def decodestring(self, code, length):
        self.code = Decimal(int(code,2))/Decimal(2**len(code))
        s = ''
        for i in range(length):
            s += self.decode()
        return s
```

- V kombinaci s vhodným prediktorem (zde PPM) je možné dále zvýšit kompresní účinnost
- Příklad:
 - Uvažujme řetězec 41 znaků
abc
 - Aritmetické kódování: 48 bitů + tabulka četností
 - PPM řádu 2: 26 bitů

```
> msg = "ab"*20 + "c"
> freq_table = collections.Counter(msg)
Counter({'a': 20, 'b': 20, 'c': 1})
> ArithmeticCoder(freq_table).encodestring(msg)
010011111111000000000011001100101000111110111101
(48 bits)
> PPM.encodestring(msg, 2)
00010010101010011011100011
(26 bits)
```

```
class PPM:
    ESC = 256
    EOF = 257

    def encodestring(string, N=3, symbols='abc'):
        ctxs = {None: {sym: 1 for sym in set(symbols)}}
        #uniform distribution of symbols
        ctxs[None][PPM.EOF] = 1
        ae = ArithmeticCoder(ctxs[None])

    def update(ctx, ch):
        if not ch in ctx: ctx[ch] = 0
        ctx[ch] += 1

    def encode(ch, freq):
        if ch in freq: ae.encode(ch, freq)

    for i,ch in itertools.chain(enumerate(string),
                                [(len(string), PPM.EOF)]):
        ctx = string[max(i-N,0):i]
        while (not ctx in ctxs) or (not ch in ctxs[ctx]):
            if (not ctx in ctxs): ctxs[ctx] = {}
            encode(PPM.ESC, ctxs[ctx])
            update(ctxs[ctx], PPM.ESC)
            update(ctxs[ctx], ch)
            ctx = None if ctx == '' else ctx[1:]
        encode(ch, ctxs[ctx])
        update(ctxs[ctx], ch)
        if ctx == None:
            del(ctxs[ctx][ch])

    return ae.finish()
```

Příklad PPM řád 2

- Uvažujme řetězec 41 znaků abc

0	-	p(ESC -)=0 p(a)=1/4
1	a	p(ESC a)=0 p(ESC -)=1/2 p(b)=1/3
2	ab	p(ESC ab)=0 p(ESC b)=0 p(a -)=1/4
3	ba	p(ESC ba)=0 p(b a)=1/2
4	ab	p(a ab)=1/2
5	ba	p(b ba)=1/2
6	ab	p(a ab)=2/3
7	ba	p(b ba)=2/3
8	ab	p(a ab)=3/4
9	ba	p(b ba)=3/4
10	ab	p(a ab)=4/5
11	ba	p(b ba)=4/5
12	ab	p(a ab)=5/6
13	ba	p(b ba)=5/6
14	ab	p(a ab)=6/7
15	ba	p(b ba)=6/7
16	ab	p(a ab)=7/8
17	ba	p(b ba)=7/8
18	ab	p(a ab)=8/9
19	ba	p(b ba)=8/9
20	ab	p(a ab)=9/10
21	ba	p(b ba)=9/10
22	ab	p(a ab)=10/11
23	ba	p(b ba)=10/11
24	ab	p(a ab)=11/12
25	ba	p(b ba)=11/12
26	ab	p(a ab)=12/13
27	ba	p(b ba)=12/13
28	ab	p(a ab)=13/14
29	ba	p(b ba)=13/14
30	ab	p(a ab)=14/15
31	ba	p(b ba)=14/15
32	ab	p(a ab)=15/16
33	ba	p(b ba)=15/16
34	ab	p(a ab)=16/17
35	ba	p(b ba)=16/17
36	ab	p(a ab)=17/18
37	ba	p(b ba)=17/18
38	ab	p(a ab)=18/19
39	ba	p(b ba)=18/19
40	ab	p(ESC ab)=1/20 p(ESC b)=1/2 p(ESC -)=2/5 p(c)=1/2
41	bc	p(ESC bc)=0 p(ESC c)=0 p(ESC -)=3/7 p(EOF)=1/1



Asymmetric Numeral Systems (ANS)

- Princip ANS (Jaroslav Duda, 2007-2014) je velmi podobný tomu, jak funguje aritmetické kódování, ale místo práce s intervalem, se pracuje s celočíselným stavem, který je aktualizován s každým načteným symbolem.
 - Jedna stavová proměnná místo dvou hranic intervalu
 - Symboly s různou pravděpodobností přispívají různě k rychlosti růstu hodnoty stavu.
 - Stav typicky omezen na pevnou bitovou šířku, při renormalizaci se zapisují bity do výstupu
- Kompresní účinností je technika srovnatelná s aritmetickým kódováním (také se snaží používat pravděpodobnosti aniž by došlo k aproximaci), avšak s výkonností Huffmanova kódování (lze implementovat pomocí tabulky).
- Algoritmy
 - uABS (2006) – binární vstupní abeceda
 - rANS (2013) – stav se aktualizuje výpočtem, vhodné pro velké vstupní abecedy
 - tANS (2007) – stav je určen náhledem do tabulky a to vč. renormalizace, vhodné pro malé abecedy
- Implementace
 - Facebook Zstandard, Apple LZFSE, NVIDIA nvCOMP (2022), Dropbox DivANS, JPEG XL (redukce 1/3)
<https://encode.su/threads/2078-List-of-Asymmetric-Numeral-Systems-implementations>



Motivace

- Uvažujme vstupní abecedu s dvěma symboly $\Sigma = \{a, b\}$, $P(a) = 0.01$, $P(b) = 0.99$

Huffmanovo kódování (1952)

- rychlé (>300MB/s/core)
- vyžaduje řazení, algoritmus bez násobení
- aproximuje $P(s) \sim 2^{-r}$ (0.5)

Aritmetické kódování (~1976)

- pomalé (>100MB/s/core)
- vyžaduje násobení
- snaha zachytit $P(s)$ co nejpřesněji

Rychlost dekódování/core Intel i7

rok 2013

rok 2016

přibližné algoritmy (Huffman)

~ 300 MB/s

~ 1000 MB/s

tA přesné algoritmy (aritm. kódování, ANS)

~ 50 MB/s

~ 1500 MB/s

- rychlé (>300MB/s/core)
- vhodné pro malé modely, fixní distribuce
- umožňuje šifrování
- snaha zachytit $P(s)$ co nejpřesněji
- ~**0.08** bit/symbol

- rychlé (>500MB/s/core)
- vhodné pro velké modely, adaptivní distribuce
- snaha zachytit $P(s)$ co nejpřesněji
- ~**0.08** bit/symbol

Princip kódování s využitím stavu – binární kód

- Předpokládejme binární vstupní abecedu. Mějme informaci uloženou v hodnotě x , nyní chceme přidat další bit informace $s \in \{0,1\}$. Jak bude vypadat nová hodnota x' ?
- Binární kódování: $x' = (x \ll 1) + s = 2x + s$

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
x' ($s = 0$)	0	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30
x' ($s = 1$)	1	3	5	7	9	11	13	15	17	19	21	23	25	27	29	31

- Rozklad
 - $N_{s=0} = \{0,2,4,6,8,10,12,14, \dots\}$
 - $N_{s=1} = \{1,3,5,7,9,11,13,15, \dots\}$
 - symetrické binární kódování

Princip kódování s využitím stavu – ANS

- Předpokládejme binární vstupní abecedu. Mějme informaci uloženou v hodnotě x , nyní chceme přidat další bit informace $s \in \{0,1\}$. Předpokládejme, že $P(0)=1/4$
- ANS kódování: $x' \approx x/P(s)$

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
x' ($s = 0$)	0	4	8	12	16	20	24	28	32	36	40	44	48	52	56	60
x' ($s = 1$)	1	2	3	5	6	7	9	10	11	13	14	15	17	18	19	21

$$x' = \left\lfloor \frac{x}{P(s)} \right\rfloor + s$$

- Rozklad
 - $N_{s=0} = \{0,4,8,12, \dots\}$
 - $N_{s=1} = \{1,2,3,5,6,7,9,10,11,13,14,15, \dots\}$
 - asymetrické binární kódování

Princip kódování s využitím stavu – binární kód

■ Rozklad pro BIN

N_{BIN}	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
------------------	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

■ Jak rychle roste hodnota stavu v závislosti na symbolu s ?

- $s = 0 \rightarrow +2,$
- $s = 1 \rightarrow +2$

■ Kolik hodnot stavu x v rozsahu $[0, k - 1]$ je přiřazeno určitému symbolu?

- $s = 0 \rightarrow \left\lfloor \frac{1}{2}k \right\rfloor$
- $s = 1 \rightarrow k - \left\lfloor \frac{1}{2}k \right\rfloor$

■ Hodnota stavu roste stejně rychle nezávisle na symbolu. Počet hodnot stavu je pro oba symboly shodný.

- Co tento výsledek implikuje z hlediska pravděpodobnosti $P(s)$?

$$x' \approx x/P(s)$$

Princip kódování s využitím stavu – ANS

- Rozklad pro ANS, $P(0)=1/4$

N_{ANS}	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
-----------	---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----

- Jak rychle roste hodnota stavu v závislosti na symbolu s ?
 - $s = 0 \rightarrow +4$,
 - $s = 1 \rightarrow +1/+2$, průměrně o 1.25
- Kolik hodnot stavu x v rozsahu $[0, k - 1]$ je přiřazeno určitému symbolu?
 - $s = 0 \rightarrow \left\lfloor \frac{1}{4}k \right\rfloor$
 - $s = 1 \rightarrow k - \left\lfloor \frac{1}{4}k \right\rfloor$
- Hodnota stavu roste pomaleji pro $s=1$. Počet hodnot stavu je vyšší pro $s=1$, v poměru $3/4$ ku $1/4$.
- Interpretace stavu: Hodnota $x=13$ odpovídá 10 výskytům $s=1$ a cca 4 výskytům $s=0$
 - poměr $10/14, 4/14$

Efektivita ANS kódování

- Pokud budeme předpokládat, že hodnota x obsahuje $\log_2 x$ bitů informace, pak nový stav x' odpovídá $\log_2 x'$ bitům informace.
- Aktualizace stavu pro ANS: $x' \approx x/P(s)$

$$\log_2 x' = \log_x x/P(s) = \log_2 x + \log_2 1/P(s)$$

- Množství informace přidané zakódováním symbolu s , který se vyskytuje s pravděpodobností $P(s)$ odpovídá $\log_2 1/P(s)$, tedy informačnímu obsahu dle Shannonovi definice.

Kódování a dekódování

■ Kódování:

- Inicializujeme výchozí stav $x_0 = 1$.
- Postupně aplikujeme kódovací funkci $x' = C(x, s)$ na jednotlivé symboly tak, že $x_i = C(x_{i-1}, s_i)$
- Jakmile zpracujeme všech n symbolů, uložíme stav x_n do souboru.

■ Dekódování

- Načteme stav x_n ze souboru.
- Dekódujeme postupně pomocí dekódovací funkce $(x, s) = D(x')$ jednotlivé symboly.
- Pokud narazíme na stav s hodnotou 1, dekódování ukončíme.

■ Vlastnosti:

- kódování a dekódování je založeno na vyčíslováním aritmetického výrazu
- nepotřebujeme zavádět symbol EOF nebo ukládat délku
- Kódování a dekódování musí probíhat zrcadlově (symboly dekódujeme v opačném pořadí)

Princip kódování s využitím stavu

- Mějme zakódovat vstupní sekvenci 01111, počáteční stav $x_0 = 1$
 - Binary

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15		23
x' ($s = 0$)	2	4	6	8	10	12	14	16	18	20	22	24	26	28	30		46
x' ($s = 1$)	3	5	7	9	11	13	15	17	19	21	23	25	27	29	31		47

47 = 101111

- ANS

x	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
x' ($s = 0$)	4	8	12	16	20	24	28	32	36	40	44	48	52	56	60
x' ($s = 1$)	2	3	5	6	7	9	10	11	13	14	15	17	18	19	21

18 = 10010



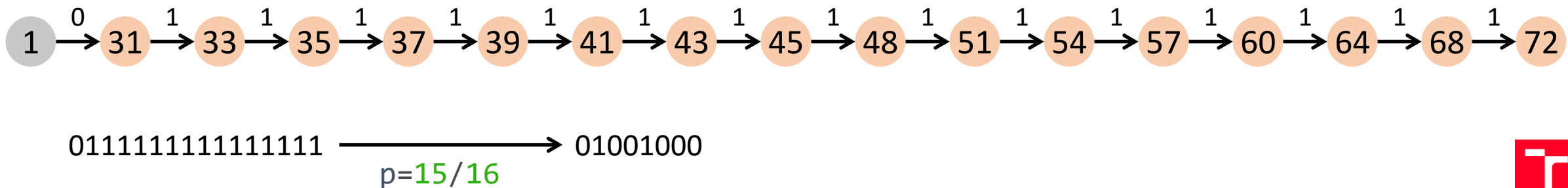
Implementace C a D

- Asymmetric binary encoding – Jarosław Duda "Optimal encoding on discrete lattice with translational invariant constraints using statistical algorithms“, kap. 3, 2007

```
def C(x, s):
    if (s == 0):
        return ceil((x+1)/(1-p)) - 1
    else:
        return floor(x/p)
```

```
def D(x):
    s = ceil((x+1)*p) - ceil(x*p)
    if s == 0:
        return x - ceil(x*p), s
    else:
        return ceil(x*p), s
```

- Parametr p odpovídá pravděpodobnosti výskytu symbolu 1
 - pro $p=0.5$ odpovídá binárnímu kódování s prohozenou 0 a 1
- Příklad:



Princip streamového zpracování v uABS

- Uvedené implementace produkují jednu velkou hodnotu a nejsou vhodné k reálnému použití, nicméně je možné přejít na celočíselnou N -bitovou aritmetiku a operaci dělení nahradit posuvem.
- Princip:
 - Pravděpodobnost p škálujeme do intervalu $q \in [1, Q - 1]$, hodnotu stavu do intervalu $x \in [Q, 2Q - 1]$, kde $Q = 2^N$ odpovídá počtu hodnot stavu.
 - Hodnota x udržována v intervalu zápisem LSB bitů x před zakódováním, a načtením do x po dekódování.
- Algoritmus:

```
x = Q
while not eof do
    read s
    while encode(x,q,s) >= 2*Q do
        write bit x mod 2
        x = floor(x/2)
    x = encode(x,q,s)
```

```
x = read N bits
while not done:
    w,d := decode(x,q)
    write d
    while w < Q do
        read bit b
        w := w*2 + b
    x := w
```

Princip uABS

- Celočíselná implementace operací encode a decode

```
def encode(x, q, s):  
    if (s == 0):  
        return ((x+1)*Q - 1)/(Q - q) - 1  
    return (x*Q)/q
```

```
def decode(x, q):  
    s = floor(((x+1)*q - 1)/Q) -  
        floor((x*q-1)/Q)  
    xq = floor((x*q - 1)/Q) + 1  
    if s == 0:  
        return x - xq, s  
    return xq, s
```

- Q je mocnina 2 => dělení nahrazeno bitovým posuvem

Varianta rANS (range ANS)

- Využívá aritmetické výrazy podobně jako uABS, ale vhodné pro velké abecedy
- Princip
 - Množina přirozených čísel se rozdělí na intervaly o velikosti 2^n prvků
 - Každý interval je rozdělen na podinterval, jejichž velikost je proporcionální k pravd. modelu.
- rANS pracuje nad kvantizovaným pravd. rozdělením (n bitů, typicky 8-12)
 - $p_s \approx f[s]/2^n$
- Hodnota stavu $y \in [0, 2^n - 1]$ je mapována na symboly následovně:
 - $\text{symbol}(y) = s$ tak, že platí $\text{CDF}[s] \leq y < \text{CDF}[s + 1]$, kde
 - $\text{CDF}[s] = \sum_{i < s} f[i] = f[0] + \dots + f[s - 1]$ je analogie kumulativní distribuční funkce s tím rozdílem, že $\text{CDF}[0]=0$, neboť prvnímu symbolu žádný nepředchází
- Kódovací a dekódovací funkce

$$C(x, s) = (\text{floor}(x/f[s]) \ll n) + (x \% f[s]) + \text{CDF}[s]$$

$$s = \text{symbol}(x \& \text{mask})$$

$$D(x) = (f[s] * (x \gg n) + (x \& \text{mask}) - \text{CDF}[s], s)$$

$$\text{mask} = 2^n - 1$$



Varianta rANS – streamová verze

- Abychom se vyhnuli práci s aritmetikou velkých čísel, v praxi se udržuje $x \in [L, b \cdot L - 1]$ pomocí renormalizace – tj. odesíláním nebo přijímáním nejméně významných bitů čísla x do nebo z bitového proudu (obvykle jsou L a b mocniny dvojky).
- Ve variantě rANS je x běžně 32-bitové číslo.
 - Pro renormalizaci na 16 bitech platí: $x \in [2^{16}, 2^{32} - 1]$
 - Dekodér doplňuje nejméně významné bity z bitového proudu, když je to potřeba s využitím následujícího schematu:

```
if (x < (1 << 16)):  
    x = (x << 16) + read16bits()
```

Princip tANS

- tANS využívá stejného principu jako předchozí metoda, avšak místo vyčíslování výrazu se užívá náhled do předpočítané tabulky
 - Stav je omezený na určitý rozsah (např. 0—1023). Bity stavu se postupně zapisují do výstupu (renormalizace).
 - Lze využít pro běžné abecedy (256 symbolů)
- Algoritmus:

```
def encode(input, p, L=1024):
    enctab, nb, start, r =
        prepareEnctab(symbProbs=p, tabLen=L)

    x = L
    for s in input:
        nbits = (x + nb[s]) >> r          #0+
        write(x, nbits)
        x = enctab[(x >> nbits) + start[s]]
    write(x - L, r-1)
```

$$\begin{aligned} nbits &\in [0, r-1] & 2^r &= 2L \\ x &\in [L, 2L-1] & L &= 2^R \end{aligned}$$

```
def decode(input, p, L=1024):
    dectab, r =
        prepareDectab(symbProbs=p, tabLen=L)

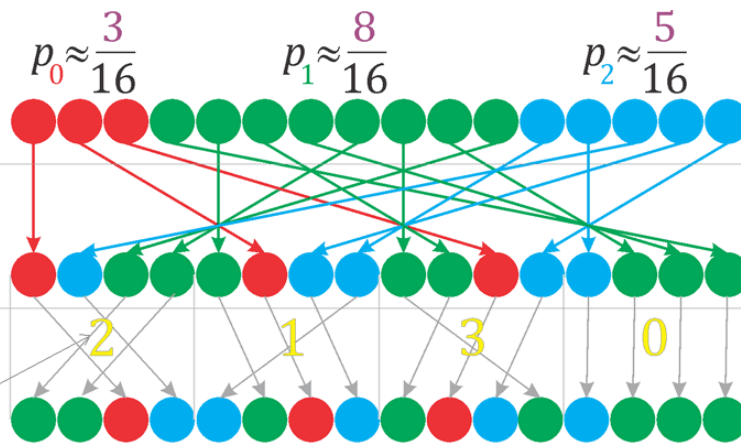
    X = read(input, r-1)
    while not eof():
        t = dectab[X]
        write(t.symbol)
        X = t.newX + read(input, t.nbits)
```

$$\begin{aligned} X &\in [0, L-1] \\ t.nbits &\in [0, r-1] \end{aligned}$$



Princip tANS – ukázka tvorby dekódovací tabulky

1. Approximate probabilities

2. Spread symbols
(fast, step = 5)2*. Scramble
(4 block cycle)

3. Enumerate

($L = 16$, $L_0 = 3$,
 $L_1 = 8$, $L_2 = 5$)

$C(s, x)$	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
$s=0$			3				4			5						
$s=1$	8	9				10			11			12		13	14	15
$s=2$				5	6			7			8		9			

4. Renormalize to remain in $I = \{L, \dots, 2L-1\}$ range

$x \rightarrow \text{newX} + \text{readBits}(\text{nbBits}) \in I$

decodingTable:

	x	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
symbol		1	1	0	2	2	1	0	2	1	0	2	1	2	1	1	1
pre-renormalization x_{tmp}		8	9	3	5	6	10	4	7	11	5	8	12	9	13	14	15
nbBits to read to return to I		1	1	3	2	2	1	2	2	1	2	1	1	1	1	1	1
$\text{newX} = x_{\text{tmp}} \ll \text{nbBits}$		16	18	24	20	24	20	16	28	22	20	16	24	18	26	28	30

5. Encode/decode - e.g. decoding 11100000011010100

bits $\rightarrow$ 11 10 00 00 011 0 1 01 0 0
 x : 25 $\xrightarrow{0}$ 23 $\xrightarrow{2}$ 30 $\xrightarrow{1}$ 28 $\xrightarrow{2}$ 18 $\xrightarrow{0}$ 27 $\xrightarrow{1}$ 24 $\xrightarrow{1}$ 23 $\xrightarrow{2}$ 29 $\xrightarrow{1}$ 26 $\xrightarrow{2}$ 16
 $s \rightarrow$ 0 2 1 2 0 1 1 2 1 2

Algoritmy tANS

Method 1 tANS decoding step, $X = x - L \in \{0, \dots, L - 1\}$

```

t = decodingTable[X]      { X ∈ {0, .., L - 1} is current state }
useSymbol(t.symbol)        { use or store decoded symbol }
X = t.newX + readBits(t.nbBits)    { state transition }

```

Method 3 Preparation for tANS decoding, $L = 2^R$

```

Require: next[s] = Ls      { number of next appearance of symbol s }
for X = 0 to L - 1 do
    t.symbol = symbol[X]      { symbol is from spread function }
    x = next[t.symbol] + +      { D(X + L) = (symbol, x) }
    t.nbBits = R - ⌊lg(x)⌋      { number of bits }
    t.newX = (x << t.nbBits) - L    { properly shift x }
    decodingTable[X] = t
end for

```

Method 2 tANS encoding step for symbol *s* and state $x = X + L$

```

nbBits = (x + nb[s]) >> r      {  $2^r = 2L$  }
useBits(x, nbBits)              { use nbBits of the youngest bits of x }
x = encodingTable[start[s] + (x >> nbBits)]

```

Method 4 Preparation for tANS encoding, $L = 2^R$, $r = R + 1$

```

Require: k[s] = R - ⌊lg(Ls)⌋      { nbBits = k[s] or k[s] - 1 }
Require: nb[s] = (k[s] << r) - (Ls << k[s])
Require: start[s] = -Ls + ∑s' < s Ls'
Require: next[s] = Ls
    for x = L to 2L - 1 do
        s = symbol[x - L]
        encodingTable[start[s] + (next[s] + +)] = x;
    end for

```

Method 7 An example of fast symbol spread function [11]

```

X = 0; step = 5/8L + 3    { some initial position and choice of step }
for s = 0 to m - 1 do
    for i = 1 to Ls do
        symbol[X] = s; X = mod(X + step, L)
    end for
end for

```

Očekávané znalosti

- Princip entropického kódování
- Princip tvorby Tunstallova kódování
- Rozumět principu konstrukce Huffmanova kódování a jeho kanonické verze, významu kanonické verze, umět zkonstruovat Huffmanův strom vč. uložení kódové knihy.
- Rozumět principu adaptivního Huffmanova kódování, umět zkonstruovat FGK a V strom pro daný vstup (provést krok algoritmu).
- Rozumět principu aritmetického kódování, umět zakódovat a dekodovat vstup pomocí základní varianty. Znát výhody oproti Huffmanovu kódování.
- Rozumět základnímu principu kódování s využitím stavu (viz ANS), umět aplikovat na binární abecedu (uABS) pokud bude zadána kódovací a dekodovací funkce